

[Deep Learning Framework]

ML 핵심 개념

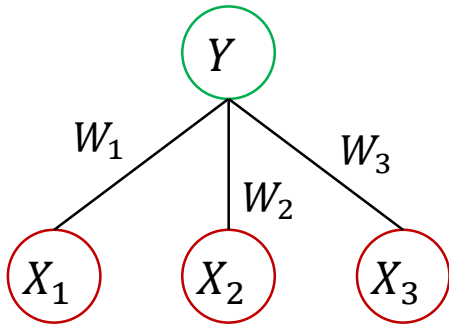
박성호

컴퓨터공학부

한국공학대학교

가장 간단한 형태의 Artificial Neural Networks (Perceptron)

- 가장 간단한 신경망 모델은 logistic regression / linear regression과 같다



For classification,

$$\hat{Y} = \text{Activation}(W_1X_1 + W_2X_2 + W_3X_3 + b)$$

For regression,

$$\hat{Y} = \hat{f}(x) = W_1X_1 + W_2X_2 + W_3X_3 + b$$

W : weight, 학습해야 할 모델의 파라미터



- 어떻게 파라미터를 찾는가?
- 종고 나뭇을 어떻게 측정할 것인가?

단순 회귀 Perceptron 학습 (파라미터 추정)

1. 비용함수 또는 목적함수 정의:

$$Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0) = \sum_i^n e_i^2 = \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x^i + \hat{\mathbf{w}}_0) \right)^2$$

2. 비용함수 또는 목적함수이용해 최적화 문제로 변환

$$\text{minimize}_{\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0} Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x^i + \hat{\mathbf{w}}_0) \right)^2$$

3. 최적화 문제 풀기



- 어떻게 파라미터를 찾는가?
- 좋고 나쁨을 어떻게 측정할 것인가?

학습이란? => 파라미터 추정 = 최적화문제 풀기

먼저, 최적화(optimization) 문제란 어떤 목적함수(objective function)의 함수값을 최적화(최대화 또는 최소화)시키는 파라미터(변수) 조합을 찾는 문제를 말합니다.

다루고자 하는 최적화 문제는 별도의 제약조건이 있는 경우와 없는 경우가 존재

$$\begin{array}{ll}\underset{x}{\text{minimize}} & f(x) \\ \text{subject to} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p\end{array}$$

where

- $f: \mathbb{R}^n \rightarrow \mathbb{R}$ is the **objective function** to be minimized over the n -variable vector x ,

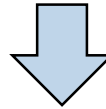
최적화(optimization) 문제는 크게 최대화(maximization) 문제와 최소화(minimization) 문제로 나눌 수 있는데 목적함수가 이윤, 점수(score) 등인 경우에는 최대화 문제, 비용(cost), 손실(loss), 에러(error) 등인 경우에는 최소화 문제

단순 회귀 Perceptron 학습 (파라미터 추정)

$$\text{minimize}_{\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0} \text{Loss}(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x^i + \hat{\mathbf{w}}_0) \right)^2$$

최적화(optimization) 문제란 어떤 목적함수(objective function) = 손실함수의 함수값을 최적화(최대화 또는 최소화)시키는 파라미터(변수) (모델 학습) 조합을 찾는 문제를 말합니다.

다루고자 하는 최적화 문제는 별도의 제약조건이 있는 경우와 없는 경우가 존재



“목적함수의 구조 및 형태와 이에 따른 찾아야 될 파라미터 형태에 따라 다양한 방법이 존재”

1. Optimization Problem Design

현실 경험 + 최적화 지식

대표적인 예는 무엇? (길찾기)

2. Solver

최적화 지식

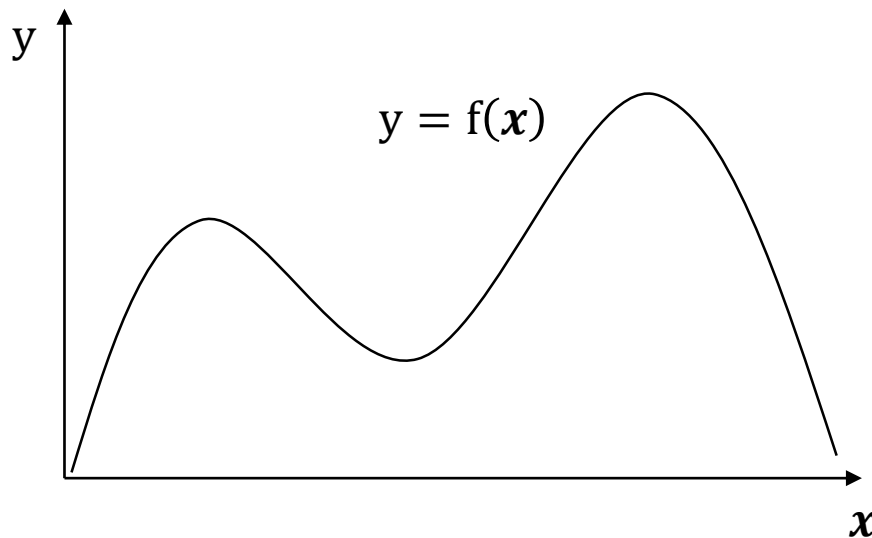
단순 회귀 Perceptron 학습 (파라미터 추정)

최적화 문제 풀기

Ordinary Least Square
Problem

$$\text{minimize}_{\hat{w}_1, \hat{w}_0} \text{Loss}(\hat{w}_1, \hat{w}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2$$

How to find the solution -> "Geometric Perspective" 이해 우선

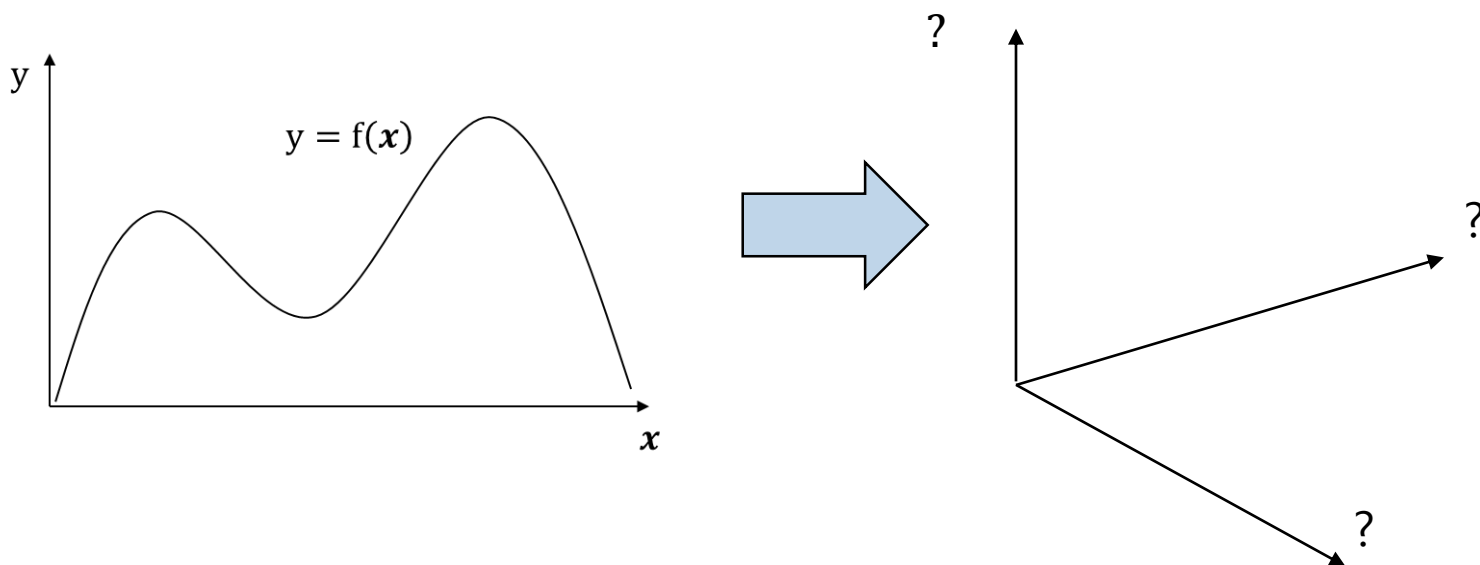


단순 회귀 Perceptron 학습 (파라미터 추정)

최적화 문제 풀기

$$\text{minimize}_{\hat{w}_1, \hat{w}_0} \text{Loss}(\hat{w}_1, \hat{w}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2$$

$$\sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2 = \sum_i^n (x^i)^2 \hat{w}_1^2 + n \hat{w}_0^2 + 2 \sum_i^n \hat{w}_0 \hat{w}_1 x^i - 2 \sum_i^n y^i x^i \hat{w}_1 - 2 \sum_i^n y^i \hat{w}_0 + \sum_i^n y^{i^2}$$



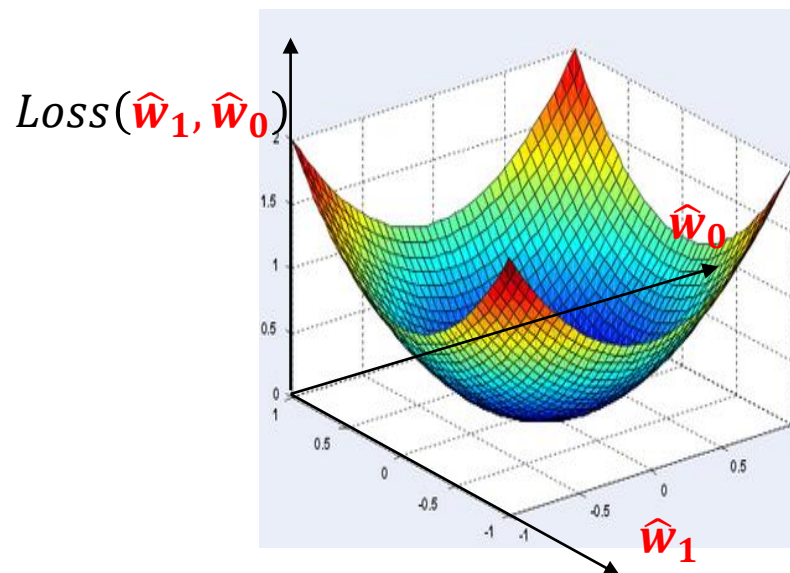
단순 회귀 Perceptron 학습 (파라미터 추정)

최적화 문제 풀기

$$\text{minimize}_{\hat{w}_1, \hat{w}_0} \text{Loss}(\hat{w}_1, \hat{w}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2$$

$$\sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2 = \sum_i^n (x^i)^2 \hat{w}_1^2 + n \hat{w}_0^2 + 2 \sum_i^n \hat{w}_0 \hat{w}_1 x^i - 2 \sum_i^n y^i x^i \hat{w}_1 - 2 \sum_i^n y^i \hat{w}_0 + \sum_i^n y^{i^2}$$

$\text{Loss}(\hat{w}_1, \hat{w}_0)$ = $\hat{w}_1, \hat{w}_0$ 에 대한 이차 함수



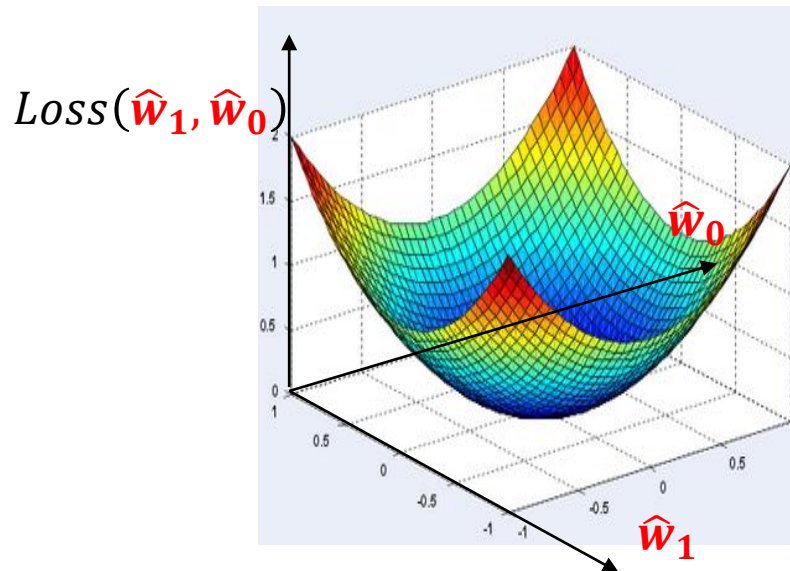
단순 회귀 Perceptron 학습 (파라미터 추정)

최적화 문제 풀기

$$\text{minimize}_{\hat{w}_1, \hat{w}_0} \text{Loss}(\hat{w}_1, \hat{w}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2$$

$$\sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2 = \sum_i^n (x^i)^2 \hat{w}_1^2 + n \hat{w}_0^2 + 2 \sum_i^n \hat{w}_0 \hat{w}_1 x^i - 2 \sum_i^n y^i x^i \hat{w}_1 - 2 \sum_i^n y^i \hat{w}_0 + \sum_i^n y^{i^2}$$

$\text{Loss}(\hat{w}_1, \hat{w}_0)$ = $\hat{w}_1, \hat{w}_0$ 에 대한 이차 함수



$$(1) \frac{\partial \text{Loss}(\hat{w}_1, \hat{w}_0)}{\partial \hat{w}_0} = 0$$

And / Or

$$(2) \frac{\partial \text{Loss}(\hat{w}_1, \hat{w}_0)}{\partial \hat{w}_1} = 0$$

단순 회귀 Perceptron 학습 (파라미터 추정)

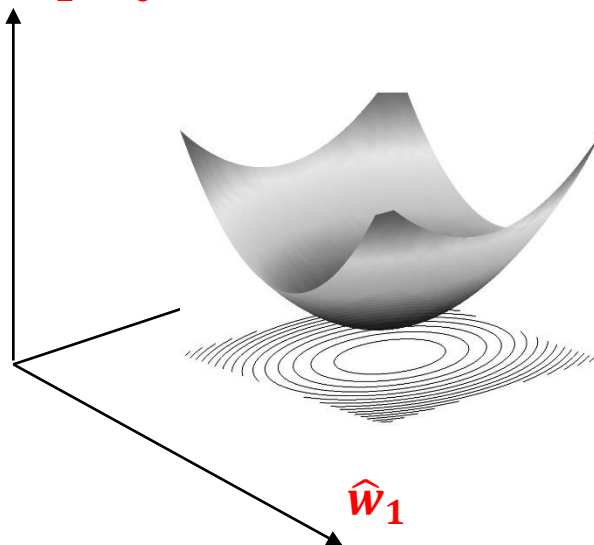
최적화 문제 풀기

$$\text{minimize}_{\hat{w}_1, \hat{w}_0} \text{Loss}(\hat{w}_1, \hat{w}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2$$

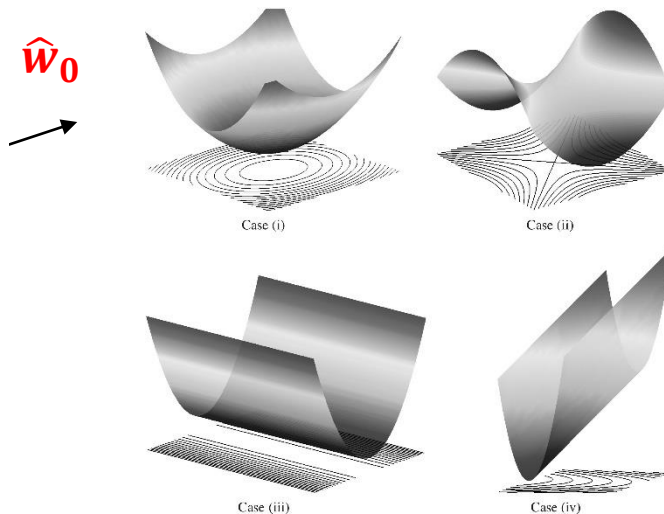
$$\sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2 = \sum_i^n (x^i)^2 \hat{w}_1^2 + n \hat{w}_0^2 + 2 \sum_i^n \hat{w}_0 \hat{w}_1 x^i - 2 \sum_i^n y^i x^i \hat{w}_1 - 2 \sum_i^n y^i \hat{w}_0 + \sum_i^n y^{i^2}$$

$\text{Loss}(\hat{w}_1, \hat{w}_0) = \hat{w}_1, \hat{w}_0$ 에 대한 이차 함수

$\text{Loss}(\hat{w}_1, \hat{w}_0)$



이차 함수 형태에 따라 접근법은 어떻게 될까?



단순 회귀 Perceptron 학습 (파라미터 추정)

$$Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0) = (\hat{\mathbf{w}}_1)^2 \sum_i^n (x^i)^2 + (\hat{\mathbf{w}}_0)^2 n + 2\hat{\mathbf{w}}_0 \hat{\mathbf{w}}_1 \sum_i^n (x^i) - 2\hat{\mathbf{w}}_0 \sum_i^n (y^i) - 2\hat{\mathbf{w}}_1 \sum_i^n (y^i x^i) + \sum_i^n (y^i)^2$$

$$(1) \frac{\partial Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_1} = (\hat{\mathbf{w}}_1) 2 \sum_i^n (x^i)^2 + 2\hat{\mathbf{w}}_0 \sum_i^n (x^i) - 2 \sum_i^n (y^i x^i) = 0$$

$$(2) \frac{\partial Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_0} = (\hat{\mathbf{w}}_0) 2n + 2\hat{\mathbf{w}}_1 \sum_i^n (x^i) - 2 \sum_i^n (y^i) = 0$$

$$(2) (\hat{\mathbf{w}}_0) + \hat{\mathbf{w}}_1 \bar{x} - \bar{y} = 0$$

$$(\hat{\mathbf{w}}_0) = \bar{y} - \hat{\mathbf{w}}_1 \bar{x}$$

$$\bar{x} = \frac{1}{n} \sum_i^n x^i$$
$$\bar{y} = \frac{1}{n} \sum_i^n y^i$$

단순 회귀 Perceptron 학습 (파라미터 추정)

$$Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0) = (\hat{\mathbf{w}}_1)^2 \sum_i^n (x^i)^2 + (\hat{\mathbf{w}}_0)^2 n + 2\hat{\mathbf{w}}_0 \hat{\mathbf{w}}_1 \sum_i^n (x^i) - 2\hat{\mathbf{w}}_0 \sum_i^n (y^i) - 2\hat{\mathbf{w}}_1 \sum_i^n (y^i x^i) + \sum_i^n (y^i)^2$$

$$(1) \frac{\partial Loss(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_1} = 2 \sum_i^n (x^i)^2 (\hat{\mathbf{w}}_1) + 2 \sum_i^n (x^i) \hat{\mathbf{w}}_0 - 2 \sum_i^n (y^i x^i) = 0$$

$$\frac{1}{n} \sum_i^n (x^i)^2 (\hat{\mathbf{w}}_1) + \bar{x} \hat{\mathbf{w}}_0 - \frac{1}{n} \sum_i^n (y^i x^i) = 0$$

$$(\hat{\mathbf{w}}_0) = \bar{y} - \hat{\mathbf{w}}_1 \bar{x}$$

$$\frac{1}{n} \sum_i^n (x^i)^2 (\hat{\mathbf{w}}_1) + (\bar{y} - \hat{\mathbf{w}}_1 \bar{x}) \bar{x} - \frac{1}{n} \sum_i^n (y^i x^i) = 0$$

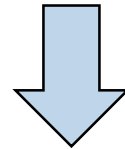
$$(\hat{\mathbf{w}}_1) \frac{1}{n} \sum_i^n (x^i)^2 + (\bar{y} \bar{x} - \hat{\mathbf{w}}_1 (\bar{x}^2)) - \frac{1}{n} \sum_i^n (y^i x^i) = 0$$

단순 회귀 Perceptron 학습 (파라미터 추정)

$$(\hat{w}_1) \frac{1}{n} \sum_i^n (x^i)^2 + (\bar{y}\bar{x} - \hat{w}_1(\bar{x}^2)) - \frac{1}{n} \sum_i^n (y^i x^i) = 0$$

$$(\hat{w}_1) \left(\frac{1}{n} \sum_i^n (x^i)^2 - \bar{x}^2 \right) = \frac{1}{n} \sum_i^n (y^i x^i) - \bar{y}\bar{x}$$

$$(\hat{w}_1) \left(\sum_i^n (x^i)^2 - n\bar{x}^2 \right) = \sum_i^n (y^i x^i) - n\bar{y}\bar{x}$$

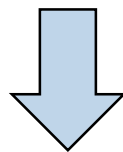


$$\hat{w}_1 = \frac{\sum_i^n (y^i x^i) - n\bar{y}\bar{x}}{(\sum_i^n (x^i)^2 - n\bar{x}^2)}$$

단순 회귀 Perceptron 학습 (파라미터 추정)

최적화 문제 풀기

$$\text{minimize}_{\hat{w}_1, \hat{w}_0} \text{Loss}(\hat{w}_1, \hat{w}_0) = \text{minimize} \sum_i^n \left(y^i - (\hat{w}_1 x^i + \hat{w}_0) \right)^2$$



미분과 연립 방정식

최적화 해

$$\hat{w}_1 = \frac{\sum_i^n (y^i x^i) - n \bar{y} \bar{x}}{(\sum_i^n (x^i)^2 - n \bar{x}^2)}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

기초수학

경영과학을 위한 기초 수학 II

2020년 6월

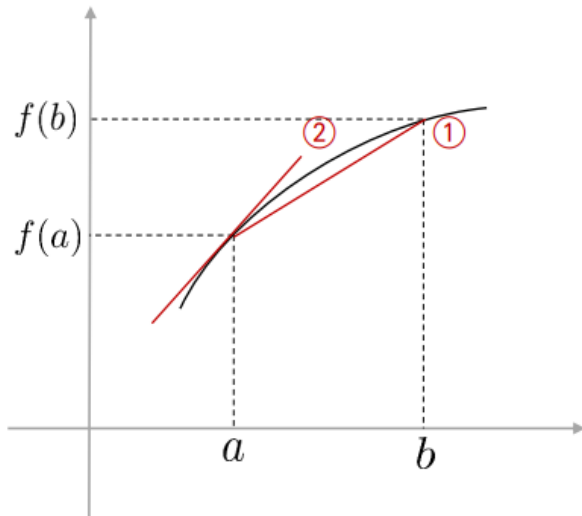
경영과학

허재석

미분 (Derivative)

미분의 기하학적 의미

■ 평균변화율과 순간변화율



① 평균변화율

= 두 점 사이의 기울기

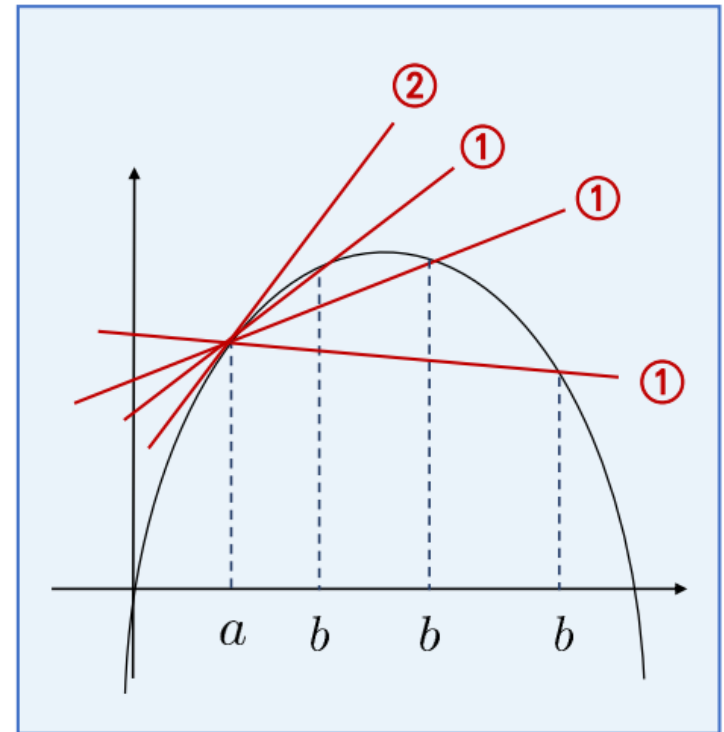
$$= \frac{f(b) - f(a)}{b - a}$$

② 순간변화율

= 한 점에서의 기울기

= 접선의 기울기

$$= \lim_{b \rightarrow a} \frac{f(b) - f(a)}{b - a}$$



미분 (Derivative)

미분계수와 도함수

■ 미분계수

- 정확히 한 점에서의 기울기
- 접선의 기울기
- 한점을 향해가는 기울기의 극한값

$$\begin{aligned} f'(a) &= \lim_{b \rightarrow a} \frac{f(b) - f(a)}{b - a} \\ &= \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h} \\ &= \lim_{h \rightarrow 0} \frac{f(a+3h) - f(a-2h)}{5h} \end{aligned}$$

한 점을 향해서 감

분자의 정의역 차이가 그대로 분모에

■ 도함수

- 미분계수를 일반화 시킨 개념
- 함수 $y = f(x)$ 가 모든 x 에서 미분 가능할 때,

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

위의 식이 정의되며, $f(x)$ 의 도함수라고 함

도함수의 여러가지 표기법

$$y' \quad \frac{dy}{dx} \quad \frac{d}{dx} f(x) \quad f'(x)$$

미분 (Derivative)

도함수 유도

- $y = f(x) = x^n$

- $$f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^n - x^n}{h} = \lim_{h \rightarrow 0} \frac{(\cancel{x} + \cancel{h} - \cancel{x})\{(x+h)^{n-1} + (x+h)^{n-2}x + \dots + x^{n-1}\}}{\cancel{h}} = nx^{n-1}$$

- $y = h(x) = f(g(x))$: 합성함수의 미분

- $$\begin{aligned} h'(x) &= \lim_{h \rightarrow 0} \frac{f(g(x+h)) - f(g(x))}{h} \\ &= \lim_{h \rightarrow 0} \frac{f(g(x+h)) - f(g(x))}{g(x+h) - g(x)} \cdot \frac{g(x+h) - g(x)}{h} \\ &= f'(g(x)) \cdot g'(x) \end{aligned}$$

각 항을 미분계수(도함수)의 정의를 따르도록 바꾸어줌

미분 (Derivative)

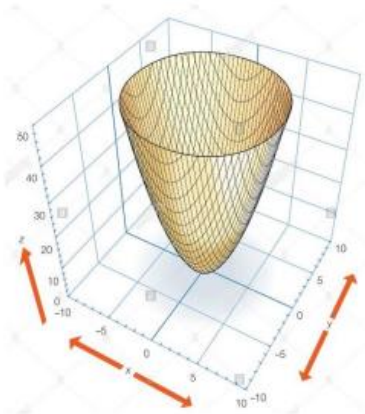
다변수 함수

- 다변수 함수의 정의

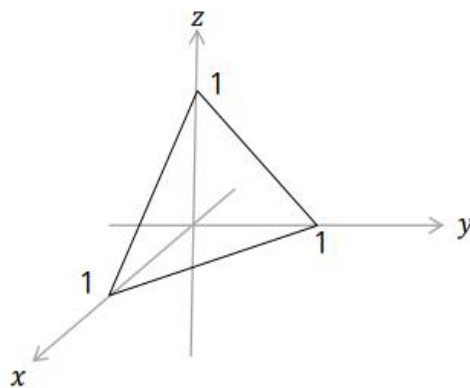
- 둘 이상의 독립변수를 갖는 함수
- $y = f(x_1, x_2, \dots, x_n)$

- 다변수 함수의 예시

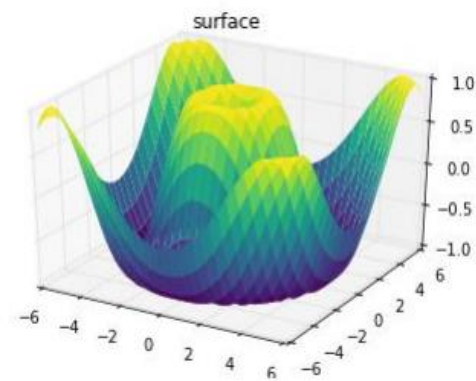
- $z = f(x, y) = x^2 + y^2$



- $z = f(x, y) = 1 - x - y$



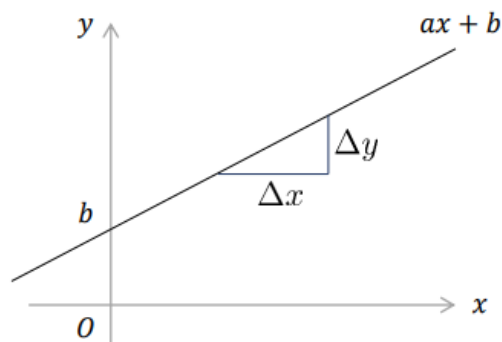
- $z = f(x, y) = \sin(\sqrt{x^2 + y^2})$



미분 (Derivative)

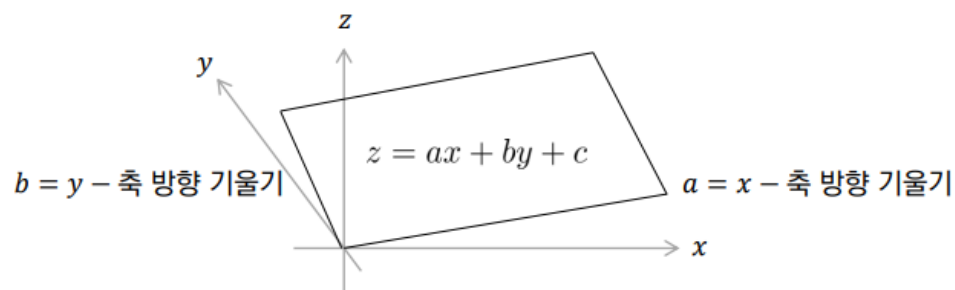
일변수함수와 다변수함수의 기울기

- 일변수함수에서의 일차함수 $y = ax + b$ 의 기울기



$$\frac{\Delta y}{\Delta x} = \frac{(a(x+\Delta x)+b)-(ax+b)}{\Delta x} = a$$

- 다변수함수에서의 일차함수 $z = f(x, y) = ax + by + c$



$$\frac{f(x+\Delta x, y) - f(x, y)}{\Delta x} = \frac{(a(x+\Delta x) + by + c) - (ax + by + c)}{\Delta x} = a$$

$$\frac{f(x, y+\Delta y) - f(x, y)}{\Delta y} = \frac{(ax + b(y+\Delta y) + c) - (ax + by + c)}{\Delta y} = b$$

- 이변수 일차함수 $z = ax + by + c$ 의 기울기는 **x-축 방향** 기울기와 **y-축 방향** 기울기로 이루어진 벡터 (a, b)

미분 (Derivative)

편도함수

- n -공간에는 n 개의 특별한 방향이 존재함: n 개의 축 방향 단위벡터인 표준단위벡터

$$e_1 = (1, 0, \dots, 0), \quad \dots, \quad e_n = (0, 0, \dots, 1)$$

- 점 $P = (p_1, \dots, p_n)$ 에서 함수 f 의 e_k -방향미분계수 $D_{e_k}f(p)$ 를 단순히 아래와 같이 씀

$D_k f(P)$: 점 P 에서 f 의 k 번째 편미분계수 (편미분계수의 여러가지 표현법: $\frac{\partial f}{\partial x_k}, D_k f$)

- k 번째 편미분계수 $\frac{\partial f}{\partial x_k}$ 는 k 번째 성분만을 변수로 하고, 나머지 변수는 상수로 보았을 때 얻은 미분계수

- 예시1) 점 $P = (1, 2, 3)$ 에서의 함수 $f(x, y, z) = x^2 y^3 + z$ 의 편미분계수 계산

$$D_1 f(P) = \frac{\partial f}{\partial x}(P) = 2xy^3(P) = 16$$

$$D_2 f(P) = \frac{\partial f}{\partial y}(P) = 3x^2 y^2(P) = 12$$

$$D_3 f(P) = \frac{\partial f}{\partial z}(P) = 1$$

-> 점 P 에서의 기울기 벡터: $(16, 12, 1)$

기울기벡터

기울기벡터

- n 변수함수 f 가 점 P 에서 모든 편미분계수가 존재할 때, f 는 점 P 에서 **편미분 가능**하다고 함
- 이때, 다음의 벡터를 P 에서의 f 의 **그래디언트 벡터(Gradient vector)** 혹은 **기울기벡터**라고 함

$$\text{grad } f(P) = (D_1 f(P), D_2 f(P), \dots, D_n f(P))$$

- 일변수함수 $y = ax + b$ 의 그래프에서 '기울기'가 a 이듯이,
다변수함수 $y = a_1 x_1 + \dots + a_n x_n + b$ 의 그래프에서 '기울기 벡터'는 $(a_1, \dots, a_n)$
- $\text{grad } f$ 는 ∇f 로 표현하기도 함 $\rightarrow \nabla = (D_1, \dots, D_n) = (\frac{\partial}{\partial x_1}, \dots, \frac{\partial}{\partial x_n})$
 - 각 방향에 대해서 편미분을 한다는 의미

- 일변수함수 $f(x)$ 의 그래프에서 접선의 기울기 $\rightarrow f'(x)$
- 다변수함수 $f(X)$ 의 그래프에서 **'접평면'의 기울기벡터** $\rightarrow \text{grad } f(X)$

기울기벡터

기울기벡터

- n 변수함수 f 가 점 P 에서 모든 편미분계수가 존재할 때, f 는 점 P 에서 **편미분 가능**하다고 함
- 이때, 다음의 벡터를 P 에서의 f 의 **그래디언트 벡터(Gradient vector)** 혹은 **기울기벡터**라고 함

$$\text{grad } f(P) = (D_1 f(P), D_2 f(P), \dots, D_n f(P))$$

- 일변수함수 $y = ax + b$ 의 그래프에서 ‘기울기’가 a 이듯이,
다변수함수 $y = a_1 x_1 + \dots + a_n x_n + b$ 의 그래프에서 ‘기울기 벡터’는 $(a_1, \dots, a_n)$
- $\text{grad } f$ 는 ∇f 로 표현하기도 함 $\rightarrow \nabla = (D_1, \dots, D_n) = (\frac{\partial}{\partial x_1}, \dots, \frac{\partial}{\partial x_n})$
 - 각 방향에 대해서 편미분을 한다는 의미

- 일변수함수 $f(x)$ 의 그래프에서 접선의 기울기 $\rightarrow f'(x)$
- 다변수함수 $f(X)$ 의 그래프에서 **‘접평면’의 기울기벡터** $\rightarrow \text{grad } f(X)$

Perceptron - 경사하강법

Numerical Solution

From the perspective of the gradient information in optimization, popular optimization methods can be divided into **three categories**:

1. First-order (Gradient Decent) optimization methods 경사하강법

$$w_{k+1} = w_k - \alpha \cdot \text{Loss}^{(1)}(w_k)$$

2. High-order optimization methods (ex: Newton's method)

$$w_{k+1} = w_k - \frac{\text{Loss}^{(1)}(x_k)}{\text{Loss}^{(2)}(x_k)}$$

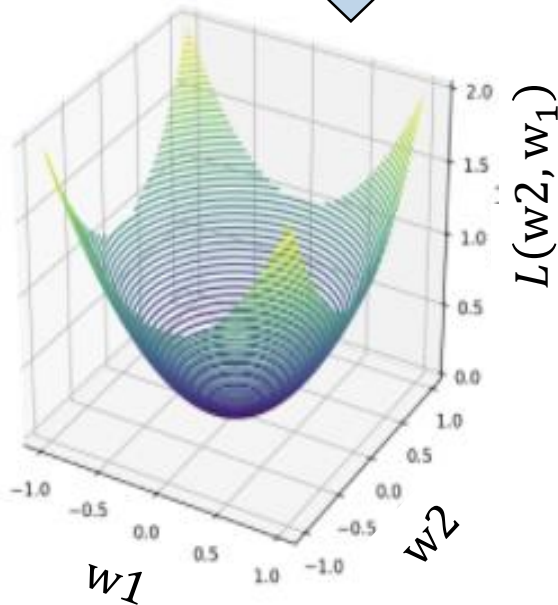
3. Heuristic derivative-free optimization methods

Gradient Descent (경사하강법)

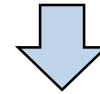
$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \hat{y}_i)^2$$



$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\})^2$$

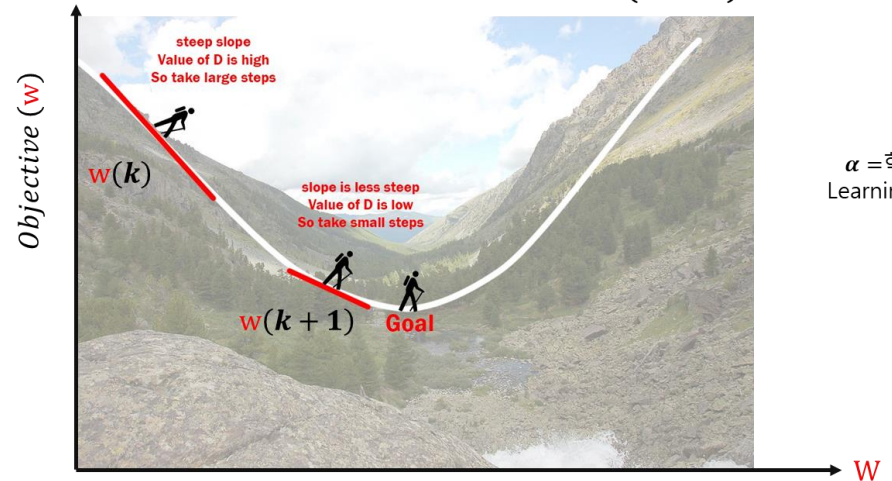


$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \hat{w}_1 x_i)^2$$



1. First-order optimization methods (Gradient Decent)

$$w(k+1) := w(k) - \alpha \cdot \text{Loss}^{(1)}(w(k))$$



α = 학습률
Learning Rate

Gradient Descent (경사하강법)

$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \hat{y}_i)^2$$

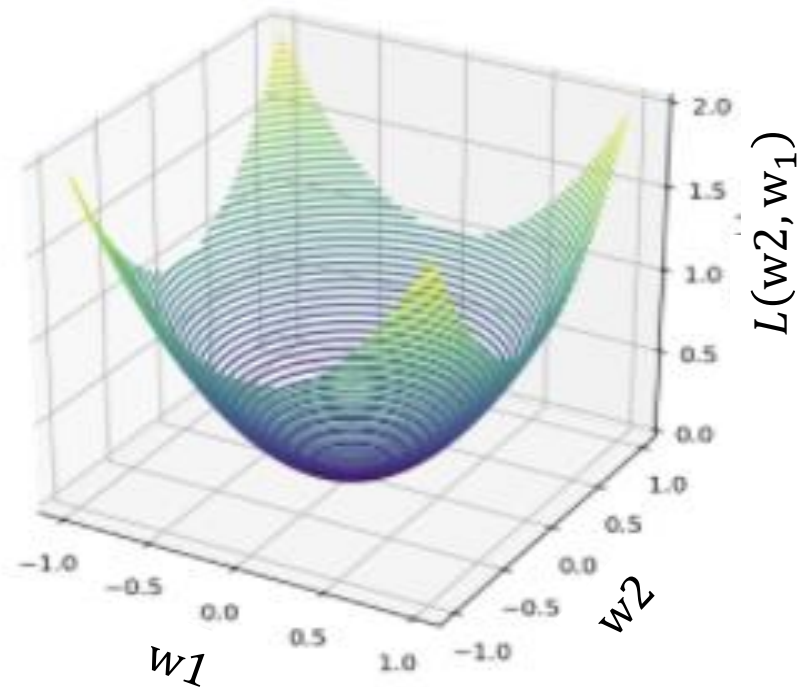


$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\})^2$$

$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \begin{bmatrix} \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_0} \\ \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_1} \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{j=1}^m -(y_i - \{\hat{w}_0 + \hat{w}_1 x_i\}) \\ \sum_{j=1}^m -x_i (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\}) \end{bmatrix}$$

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \frac{\partial \text{Loss}(\hat{\mathbf{w}}(k))}{\partial \hat{\mathbf{w}}(k)}$$

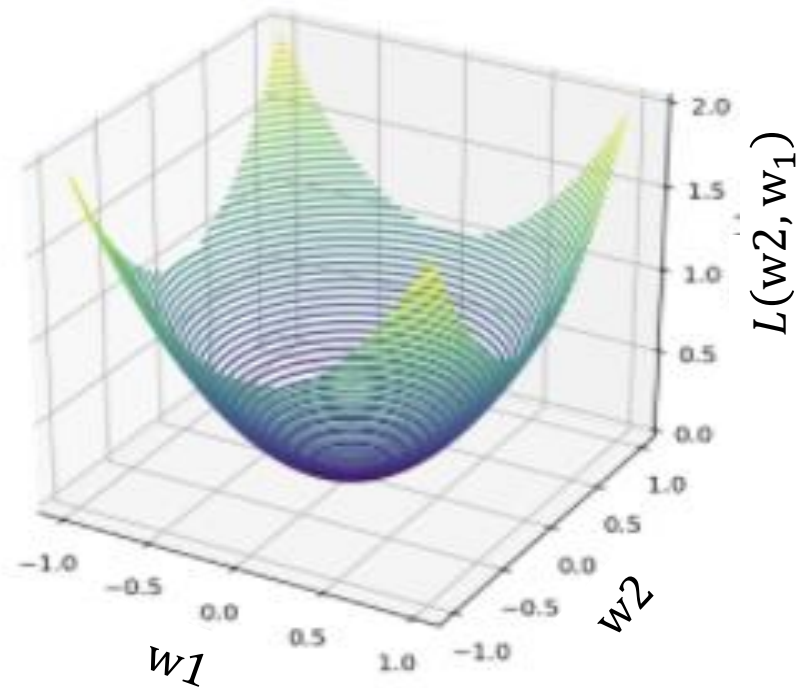


Gradient Descent (경사하강법)

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \frac{\partial \text{Loss}(\hat{\mathbf{w}}(k))}{\partial \hat{\mathbf{w}}(k)}$$

1. 가중치 (학습 파라미터) 임의로 생성
2. 데이터의 Loss에 대해 Gradient (미분)계산
3. 반복
 - 언제까지? 미분값 0
 - 적절한 횟수 만큼 = epoch

$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \begin{bmatrix} \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_0} \\ \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_1} \end{bmatrix}$$
$$= \begin{bmatrix} \sum_{j=1}^m -(y_i - \{\hat{w}_0 + \hat{w}_1 x_i\}) \\ \sum_{j=1}^m -x_i (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\}) \end{bmatrix}$$



Gradient Descent (경사하강법)

$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \begin{bmatrix} \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_0} \\ \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_1} \end{bmatrix}$$

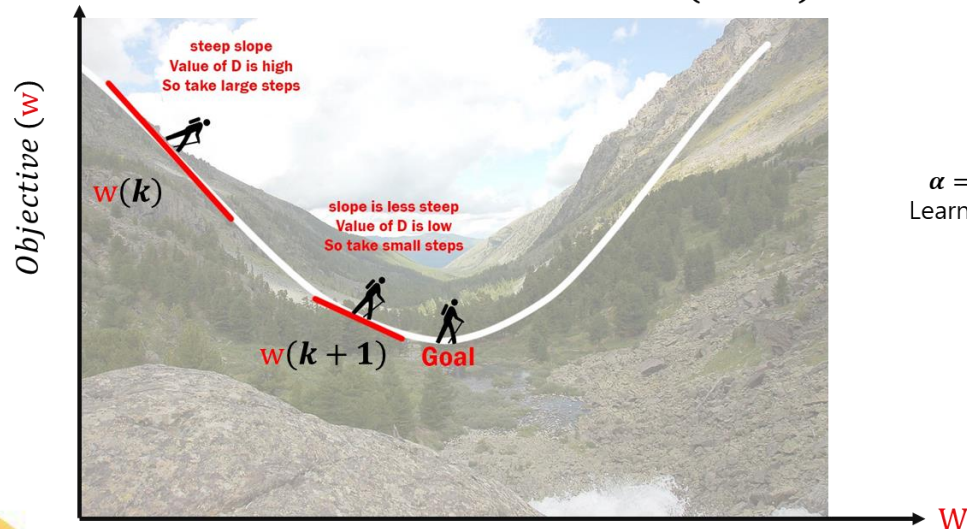
$$= \begin{bmatrix} \sum_{j=1}^m -(y_i - \{\hat{y}_i\}) \\ \sum_{j=1}^m -x_i(y_i - \{\hat{y}_i\}) \end{bmatrix}$$

What does mean?



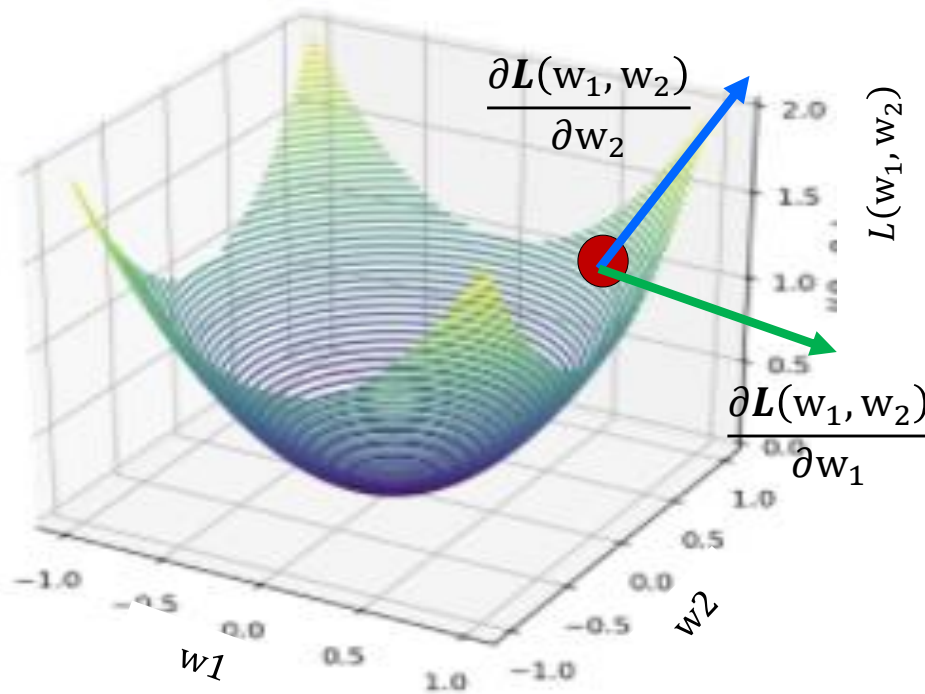
1. First-order optimization methods (Gradient Decent)

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \text{Loss}^{(1)}(\mathbf{w}(k))$$



Derivative & Directional Derivative & Gradient

Given $z = \text{Loss}(w_1, w_2)$



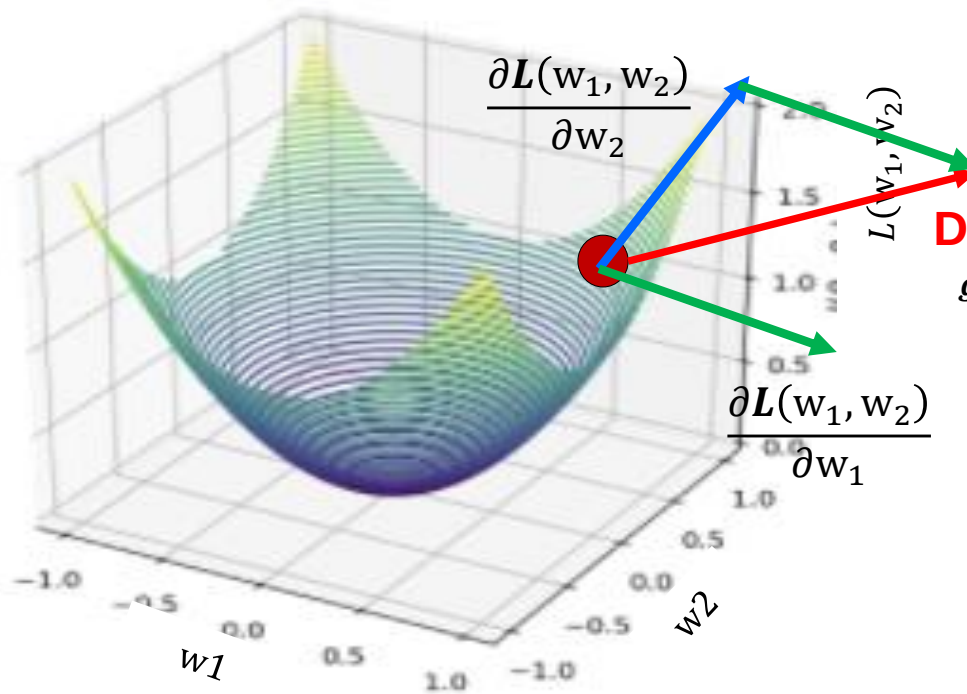
인풋 변화에 따른 아웃풋 **변화량**
인풋 -> Multivariate -> 방향 -> ??

$$\text{gradient} = \nabla L(w_1, w_2) = \left(\frac{\partial L(w_1, w_2)}{\partial w_1}, \frac{\partial L(w_1, w_2)}{\partial w_2} \right)$$

편미분(Partial derivative): 둘이상의 변수를 가지는 함수가 있을 경우 각각의 변수에 대해 독립적으로 미분. 하나에 대해 미분 나머지는 상수 취급

Derivative & Directional Derivative & Gradient

Given $z = \text{Loss}(w_1, w_2)$



인풋 변화에 따른 아웃풋 변화량
인풋 -> Multivariate -> 방향 ->??

Direction

$$\text{gradient} = \nabla L(w_1, w_2) = \left(\frac{\partial L(w_1, w_2)}{\partial w_1}, \frac{\partial L(w_1, w_2)}{\partial w_2} \right)$$

편미분(Partial derivative): 둘이상의 변수를 가지는 함수가 있을 경우 각각의 변수에 대해 독립적으로 미분. 하나에 대해 미분 나머지는 상수 취급

Directional Derivative & Gradient

In [mathematics](#), the **directional derivative** of a multivariable [differentiable \(scalar\) function](#) along a given [vector](#) $\mathbf{v}$ at a given point $\mathbf{x}$ intuitively represents the instantaneous rate of change of the function, moving through $\mathbf{x}$ with a velocity specified by $\mathbf{v}$.

The directional derivative of a [scalar function](#) f with respect to a vector $\mathbf{v}$ at a point (e.g., position) $\mathbf{x}$ may be denoted by any of the following:

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = f'_{\mathbf{v}}(\mathbf{x}) = D_{\mathbf{v}} f(\mathbf{x}) = Df(\mathbf{x})(\mathbf{v}) = \partial_{\mathbf{v}} f(\mathbf{x}) = \mathbf{v} \cdot \nabla f(\mathbf{x}) = \mathbf{v} \cdot \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}.$$

- $\mathbf{v}$ is a unit vector

Definition [\[edit \]](#)

The *directional derivative* of a [scalar function](#)

$$f(\mathbf{x}) = f(x_1, x_2, \dots, x_n)$$

along a vector

$$\mathbf{v} = (v_1, \dots, v_n)$$

is the [function](#) $\nabla_{\mathbf{v}} f$ defined by the [limit](#)^[1]

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = \lim_{h \rightarrow 0} \frac{f(\mathbf{x} + h\mathbf{v}) - f(\mathbf{x})}{h}.$$

This definition is valid in a broad range of contexts, for example where the [norm](#) of a vector (and hence a unit vector) is undefined.^[2]

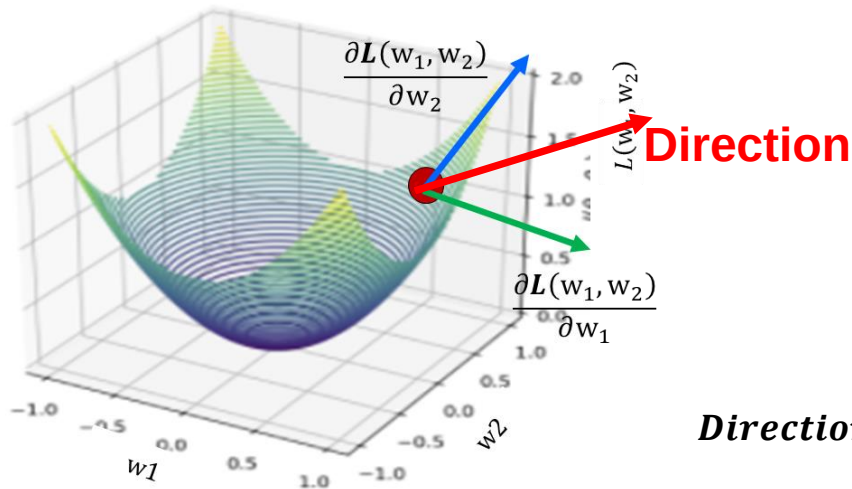
Directional Derivative & Gradient

In mathematics, the **directional derivative** of a multivariable differentiable (scalar) function along a given vector $\mathbf{v}$ at a given point $\mathbf{x}$ intuitively represents the instantaneous rate of change of the function, moving through $\mathbf{x}$ with a velocity specified by $\mathbf{v}$.

The directional derivative of a scalar function f with respect to a vector $\mathbf{v}$ at a point (e.g., position) $\mathbf{x}$ may be denoted by any of the following:

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = f'_{\mathbf{v}}(\mathbf{x}) = D_{\mathbf{v}} f(\mathbf{x}) = Df(\mathbf{x})(\mathbf{v}) = \partial_{\mathbf{v}} f(\mathbf{x}) = \mathbf{v} \cdot \nabla f(\mathbf{x}) = \mathbf{v} \cdot \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}.$$

- $\mathbf{v}$ is a unit vector



$$\text{gradient} = \nabla L(w_1, w_2) = \left(\frac{\partial L(w_1, w_2)}{\partial w_1}, \frac{\partial L(w_1, w_2)}{\partial w_2} \right) = (1, 2)$$

$$\mathbf{v} = \left(\frac{1}{\sqrt{5}}, \frac{2}{\sqrt{5}} \right)$$

$$\text{Directional Derivative } \nabla_{\mathbf{v}} L(w_1, w_2) = \frac{5}{\sqrt{5}}$$

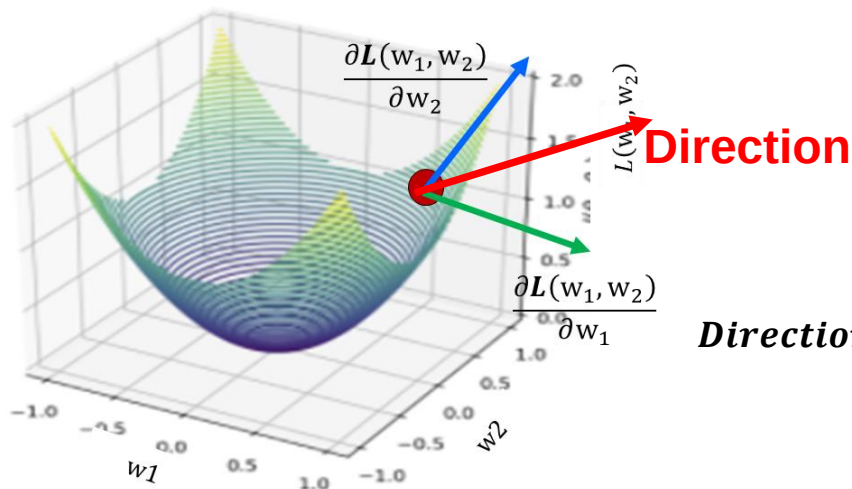
Directional Derivative & Gradient

In mathematics, the **directional derivative** of a multivariable differentiable (scalar) function along a given vector $\mathbf{v}$ at a given point $\mathbf{x}$ intuitively represents the instantaneous rate of change of the function, moving through $\mathbf{x}$ with a velocity specified by $\mathbf{v}$.

The directional derivative of a scalar function f with respect to a vector $\mathbf{v}$ at a point (e.g., position) $\mathbf{x}$ may be denoted by any of the following:

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = f'_{\mathbf{v}}(\mathbf{x}) = D_{\mathbf{v}} f(\mathbf{x}) = Df(\mathbf{x})(\mathbf{v}) = \partial_{\mathbf{v}} f(\mathbf{x}) = \mathbf{v} \cdot \nabla f(\mathbf{x}) = \mathbf{v} \cdot \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}.$$

- $\mathbf{v}$ is a unit vector



$$\begin{aligned} \text{gradient} = \nabla L(w_1, w_2) &= \left(\frac{\partial L(w_1, w_2)}{\partial w_1}, \frac{\partial L(w_1, w_2)}{\partial w_2} \right) \\ &= (1, 2) \end{aligned}$$

$$\mathbf{v} = (1, 0)$$

$$\text{Directional Derivative } \nabla_{\mathbf{v}} L(w_1, w_2) = 1 = \frac{\partial L(w_1, w_2)}{\partial w_1}$$

$$\mathbf{v} = (0, 1)$$

$$\text{Directional Derivative } \nabla_{\mathbf{v}} L(w_1, w_2) = 2 = \frac{\partial L(w_1, w_2)}{\partial w_2}$$

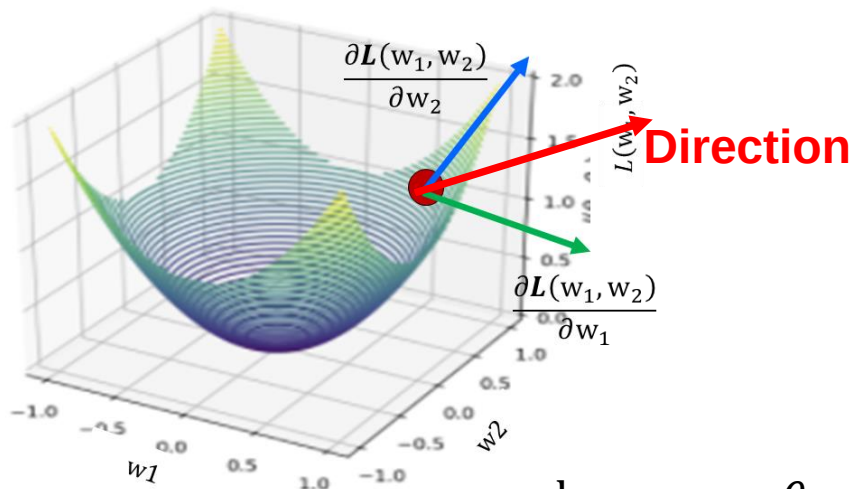
Directional Derivative & Gradient

In mathematics, the **directional derivative** of a multivariable differentiable (scalar) function along a given vector $\mathbf{v}$ at a given point $\mathbf{x}$ intuitively represents the instantaneous rate of change of the function, moving through $\mathbf{x}$ with a velocity specified by $\mathbf{v}$.

The directional derivative of a scalar function f with respect to a vector $\mathbf{v}$ at a point (e.g., position) $\mathbf{x}$ may be denoted by any of the following:

$$\nabla_{\mathbf{v}} f(\mathbf{x}) = f'_{\mathbf{v}}(\mathbf{x}) = D_{\mathbf{v}} f(\mathbf{x}) = Df(\mathbf{x})(\mathbf{v}) = \partial_{\mathbf{v}} f(\mathbf{x}) = \mathbf{v} \cdot \nabla f(\mathbf{x}) = \mathbf{v} \cdot \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}}.$$

- $\mathbf{v}$ is a unit vector



$$\begin{aligned} \text{gradient} = \nabla L(w_1, w_2) &= \left(\frac{\partial L(w_1, w_2)}{\partial w_1}, \frac{\partial L(w_1, w_2)}{\partial w_2} \right) \\ &= (1, 2) \end{aligned}$$

$$\mathbf{v} = (v_1, v_2)$$

$$\mathbf{v} \cdot \nabla L(w_1, w_2) = |\mathbf{v}| |\nabla L(w_1, w_2)| \cos \theta$$

when $\cos \theta = 1$

when $\cos \theta = -1$



What ??

다중 회귀 Perceptron with Normal Equation

1. 비용함수 또는 목적함수 정의:

$$Loss(\hat{\mathbf{w}}) = \sum_i^n e_i^2 = \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x_1^i + \hat{\mathbf{w}}_2 x_2^i + \hat{\mathbf{w}}_0) \right)^2$$

Y	X1	X2	$\hat{f}(X)$	e
175	170	3	$\hat{\mathbf{w}}_1 170 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0$	$175 - (\hat{\mathbf{w}}_1 170 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0)$
160	165	5	$\hat{\mathbf{w}}_1 165 + \hat{\mathbf{w}}_2 5 + \hat{\mathbf{w}}_0$	$165 - (\hat{\mathbf{w}}_1 165 + \hat{\mathbf{w}}_2 5 + \hat{\mathbf{w}}_0)$
181	180	1	$\hat{\mathbf{w}}_1 180 + \hat{\mathbf{w}}_2 1 + \hat{\mathbf{w}}_0$	$181 - (\hat{\mathbf{w}}_1 180 + \hat{\mathbf{w}}_2 1 + \hat{\mathbf{w}}_0)$
177	175	2	$\hat{\mathbf{w}}_1 175 + \hat{\mathbf{w}}_2 2 + \hat{\mathbf{w}}_0$	$177 - (\hat{\mathbf{w}}_1 175 + \hat{\mathbf{w}}_2 2 + \hat{\mathbf{w}}_0)$
156.5	154	3	$\hat{\mathbf{w}}_1 154 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0$	$156.5 - (\hat{\mathbf{w}}_1 154 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0)$
166	165.3	4	$\hat{\mathbf{w}}_1 165.3 + \hat{\mathbf{w}}_2 4 + \hat{\mathbf{w}}_0$	$166 - (\hat{\mathbf{w}}_1 165.3 + \hat{\mathbf{w}}_2 4 + \hat{\mathbf{w}}_0)$
....	...	...	...	...

$$Y = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \end{bmatrix} \quad X = \begin{bmatrix} x_1^1 & x_2^1 & 1 \\ x_1^2 & x_2^2 & 1 \\ \vdots & \vdots & \dots \end{bmatrix} = \begin{bmatrix} x_1^1 & x_2^1 & \dots \\ x_1^2 & x_2^2 & \dots \\ \vdots & \vdots & \dots \end{bmatrix} \quad \hat{\mathbf{w}} = \begin{bmatrix} \hat{\mathbf{w}}_1 \\ \hat{\mathbf{w}}_2 \\ \hat{\mathbf{w}}_0 \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{w}}_1 \\ \hat{\mathbf{w}}_2 \\ \vdots \end{bmatrix}$$

$n \times 1$
 $n \times (p + 1)$
 $(p + 1) \times 1$

$$X\hat{\mathbf{w}} = \begin{bmatrix} x_1^1 & x_2^1 & 1 \\ x_1^2 & x_2^2 & 1 \\ \vdots & \vdots & \dots \end{bmatrix} \begin{bmatrix} \hat{\mathbf{w}}_1 \\ \hat{\mathbf{w}}_2 \\ \hat{\mathbf{w}}_0 \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{w}}_1 x_1^1 + \hat{\mathbf{w}}_2 x_2^1 + \hat{\mathbf{w}}_0 \\ \hat{\mathbf{w}}_1 x_1^2 + \hat{\mathbf{w}}_2 x_2^2 + \hat{\mathbf{w}}_0 \\ \vdots \end{bmatrix}$$

다중 회귀 Perceptron with Normal Equation

1. 비용함수 또는 목적함수 정의:

$$Loss(\hat{\mathbf{w}}) = \sum_i^n e_i^2 = \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x_1^i + \hat{\mathbf{w}}_2 x_2^i + \hat{\mathbf{w}}_0) \right)^2$$

Y	X1	X2	$\hat{f}(X)$	e
175	170	3	$\hat{\mathbf{w}}_1 170 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0$	$175 - (\hat{\mathbf{w}}_1 170 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0)$
160	165	5	$\hat{\mathbf{w}}_1 165 + \hat{\mathbf{w}}_2 5 + \hat{\mathbf{w}}_0$	$165 - (\hat{\mathbf{w}}_1 165 + \hat{\mathbf{w}}_2 5 + \hat{\mathbf{w}}_0)$
181	180	1	$\hat{\mathbf{w}}_1 180 + \hat{\mathbf{w}}_2 1 + \hat{\mathbf{w}}_0$	$181 - (\hat{\mathbf{w}}_1 180 + \hat{\mathbf{w}}_2 1 + \hat{\mathbf{w}}_0)$
177	175	2	$\hat{\mathbf{w}}_1 175 + \hat{\mathbf{w}}_2 2 + \hat{\mathbf{w}}_0$	$177 - (\hat{\mathbf{w}}_1 175 + \hat{\mathbf{w}}_2 2 + \hat{\mathbf{w}}_0)$
156.5	154	3	$\hat{\mathbf{w}}_1 154 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0$	$156.5 - (\hat{\mathbf{w}}_1 154 + \hat{\mathbf{w}}_2 3 + \hat{\mathbf{w}}_0)$
166	165.3	4	$\hat{\mathbf{w}}_1 165.3 + \hat{\mathbf{w}}_2 4 + \hat{\mathbf{w}}_0$	$166 - (\hat{\mathbf{w}}_1 165.3 + \hat{\mathbf{w}}_2 4 + \hat{\mathbf{w}}_0)$
...	...	...	...	...

$$Y = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} x_1^1 & x_2^1 & 1 \\ x_1^2 & x_2^2 & 1 \\ \vdots & \vdots & \dots \end{bmatrix} = \begin{bmatrix} x_1^1 & x_2^1 & \dots \\ x_1^2 & x_2^2 & \dots \\ \vdots & \vdots & \dots \end{bmatrix} \quad \hat{\mathbf{w}} = \begin{bmatrix} \hat{\mathbf{w}}_1 \\ \hat{\mathbf{w}}_2 \\ \hat{\mathbf{w}}_0 \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{w}}_1 \\ \hat{\mathbf{w}}_2 \\ \vdots \end{bmatrix}$$

$n \times 1$
 $n \times (p + 1)$
 $(p + 1) \times 1$

$$Y - \mathbf{X}\hat{\mathbf{w}} = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \end{bmatrix} - \begin{bmatrix} x_1^1 & x_2^1 & 1 \\ x_1^2 & x_2^2 & 1 \\ \vdots & \vdots & \dots \end{bmatrix} \begin{bmatrix} \hat{\mathbf{w}}_1 \\ \hat{\mathbf{w}}_2 \\ \hat{\mathbf{w}}_0 \end{bmatrix} = \begin{bmatrix} y^1 - (\hat{\mathbf{w}}_1 x_1^1 + \hat{\mathbf{w}}_2 x_2^1 + \hat{\mathbf{w}}_0) \\ y^2 - (\hat{\mathbf{w}}_1 x_1^2 + \hat{\mathbf{w}}_2 x_2^2 + \hat{\mathbf{w}}_0) \\ \vdots \end{bmatrix}$$

다중 회귀 Perceptron with Normal Equation

1. 비용함수 또는 목적함수 정의:

$$Loss(\hat{\mathbf{w}}) = \sum_i^n e_i^2 = \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x_1^i + \hat{\mathbf{w}}_2 x_2^i + \hat{\mathbf{w}}_0) \right)^2$$

Y	X1	X2	$\hat{f}(X)$	e
175	170	3	$\hat{w}_1 170 + \hat{w}_2 3 + \hat{w}_0$	$175 - (\hat{w}_1 170 + \hat{w}_2 3 + \hat{w}_0)$
160	165	5	$\hat{w}_1 165 + \hat{w}_2 5 + \hat{w}_0$	$165 - (\hat{w}_1 165 + \hat{w}_2 5 + \hat{w}_0)$
181	180	1	$\hat{w}_1 180 + \hat{w}_2 1 + \hat{w}_0$	$181 - (\hat{w}_1 180 + \hat{w}_2 1 + \hat{w}_0)$
177	175	2	$\hat{w}_1 175 + \hat{w}_2 2 + \hat{w}_0$	$177 - (\hat{w}_1 175 + \hat{w}_2 2 + \hat{w}_0)$
156.5	154	3	$\hat{w}_1 154 + \hat{w}_2 3 + \hat{w}_0$	$156.5 - (\hat{w}_1 154 + \hat{w}_2 3 + \hat{w}_0)$
166	165.3	4	$\hat{w}_1 165.3 + \hat{w}_2 4 + \hat{w}_0$	$166 - (\hat{w}_1 165.3 + \hat{w}_2 4 + \hat{w}_0)$
...	...	...	...	...

$$Y = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} x_1^1 & x_2^1 & 1 \\ x_1^2 & x_2^2 & 1 \\ \vdots & \vdots & \dots \end{bmatrix} = \begin{bmatrix} x_1^1 & x_2^1 & \dots \\ x_1^2 & x_2^2 & \dots \\ \vdots & \vdots & \dots \end{bmatrix} \quad \hat{\mathbf{w}} = \begin{bmatrix} \hat{w}_1 \\ \hat{w}_2 \\ \hat{w}_0 \end{bmatrix} = \begin{bmatrix} \hat{w}_1 \\ \hat{w}_2 \\ \vdots \end{bmatrix}$$

$n \times 1$
 $n \times (p + 1)$
 $(p + 1) \times 1$

$$Loss(\hat{\mathbf{w}}) = \sum_i^n \left(y^i - (\hat{\mathbf{w}}_1 x_1^i + \hat{\mathbf{w}}_2 x_2^i + \hat{\mathbf{w}}_0) \right)^2 = (Y - \mathbf{X}\hat{\mathbf{w}})^T (Y - \mathbf{X}\hat{\mathbf{w}})$$

다중 회귀 Perceptron with Normal Equation

1. 비용함수 또는 목적함수 정의:

$$Loss(\hat{\mathbf{w}}) = (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$

2. 비용함수 또는 목적함수이용해 최적화 문제로 변환

$$\underset{\hat{\mathbf{w}}}{\text{minimize}} Loss(\hat{\mathbf{w}}) = \underset{\hat{\mathbf{w}}}{\text{minimize}} (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$

3. 최적화 문제 풀기

다중 회귀 Perceptron with Normal Equation

2. 비용함수 또는 목적함수이용해 최적화 문제로 변환

$$\text{minimize}_{\hat{\mathbf{w}}} \text{Loss}(\hat{\mathbf{w}}) = \text{minimize}_{\hat{\mathbf{w}}} (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$

3. 최적화 문제 풀기

$$\text{Loss}(\hat{\mathbf{w}}) = (\mathbf{Y}^T - \hat{\mathbf{w}}^T \mathbf{X}^T)(\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}}) =$$

$$\mathbf{Y}^T \mathbf{Y} - \mathbf{Y}^T \mathbf{X} \hat{\mathbf{w}} - \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{Y} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}}$$

$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \frac{\partial (\mathbf{Y}^T \mathbf{Y} - \mathbf{Y}^T \mathbf{X} \hat{\mathbf{w}} - \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{Y} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = 0$$

$$= \frac{\partial (-\mathbf{Y}^T \mathbf{X} \hat{\mathbf{w}} - \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{Y} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = 0$$

다중 회귀 Perceptron with Normal Equation

2. 비용함수 또는 목적함수이용해 최적화 문제로 변환

$$\text{minimize}_{\hat{\mathbf{w}}} \text{Loss}(\hat{\mathbf{w}}) = \text{minimize}_{\hat{\mathbf{w}}} (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$

3. 최적화 문제 풀기

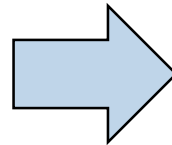
$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \frac{\partial (-\mathbf{Y}^T \mathbf{X} \hat{\mathbf{w}} - \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{Y} + \hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}}$$

$$1 \times n \quad n \times (p+1) \quad (p+1) \times 1$$

$$1 \times (p+1) \quad (p+1) \times n \quad n \times 1$$

$$\frac{\partial (\mathbf{Y}^T \mathbf{X} \hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \frac{\partial (\hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{Y})}{\partial \hat{\mathbf{w}}} = \mathbf{X}^T \mathbf{Y}$$

$$\frac{\partial (\hat{\mathbf{w}}^T \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = 2\mathbf{X}^T \mathbf{X} \hat{\mathbf{w}}$$



$$-2\mathbf{X}^T \mathbf{Y} + 2\mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = 0$$

$$\mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^T \mathbf{Y}$$

다중 회귀 Perceptron with Normal Equation

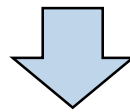
2. 비용함수 또는 목적함수이용해 최적화 문제로 변환

$$\text{minimize}_{\hat{\mathbf{w}}} \text{Loss}(\hat{\mathbf{w}}) = \text{minimize}_{\hat{\mathbf{w}}} (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$

3. 최적화 문제 풀기

$$\mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = \mathbf{X}^T \mathbf{Y}$$

$$(p+1) \times n \quad n \times (p+1)$$



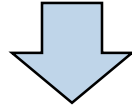
$$(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{X} \hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$$

Normal Equation

다중 회귀 Perceptron with Normal Equation

$$\text{minimize}_{\hat{\mathbf{w}}} \text{Loss}(\hat{\mathbf{w}}) = \text{minimize}_{\hat{\mathbf{w}}} (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$



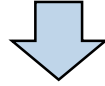
$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}, \text{ Closed Form Solution}$$

Normal Equation

- **It's slow** — having a short, nice equation does not mean that computing it is fast. Matrix multiplication is $O(n^3)$, inversion is also $O(n^3)$. This is actually slower than gradient descent for even modest sized datasets.
- **It's numerically unstable** — matrix multiplication $\mathbf{X}^T * \mathbf{X}$ squares the condition number of the matrix, and later we have to additionally multiply the result by $\mathbf{X}^T$. **This can make the results extremely unstable and this is the main reason why this method is almost never used outside pen and paper linear algebra or statistics courses.** Not even solving with Cholesky decomposition will save this.
- **This method should never be used in practice in machine learning. It is nice for mathematical analysis, but that's it. However, it has become the basis for methods that are actually used by Scikit-learn and other libraries.**

다중 회귀 Perceptron with Normal Equation

$$\text{minimize}_{\hat{\mathbf{w}}} \text{Loss}(\hat{\mathbf{w}}) = \text{minimize}_{\hat{\mathbf{w}}} (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{Y} - \mathbf{X}\hat{\mathbf{w}})$$



- $\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$ is Unstable solution, what does it mean?
- Not calculating $(\mathbf{X}^T \mathbf{X})^{-1}$ means that the determinant of $(\mathbf{X}^T \mathbf{X})^{-1}$ equal to zero
- This indicates that $\mathbf{X}^T \mathbf{X}$ is a singular matrix
- In this situation, we cannot solve linear equations $(\mathbf{X}^T \mathbf{X})\hat{\mathbf{w}} = \mathbf{X}^T \mathbf{Y}$
- What is $\mathbf{X}^T \mathbf{X}$? => Covariance Matrix

다중 회귀 Perceptron with Normal Equation

- What is $X^T X$? => Covariance Matrix

$$\begin{bmatrix} \text{Var}(x_1) & \dots & \text{Cov}(x_n, x_1) \\ \vdots & & \vdots \\ \text{Cov}(x_n, x_1) & \dots & \text{Var}(x_n) \end{bmatrix}$$

Population Variance: $\text{var}(x) = \frac{\sum_1^n (x_i - \mu)^2}{n}$

Population Covariance: $\text{cov}(x, y) = \frac{\sum_1^n (x_i - \mu_x)(y_i - \mu_y)}{n}$

Sample Variance: $\text{var}(x) = \frac{\sum_1^n (x_i - \bar{x})^2}{n-1}$

Sample Covariance: $\text{cov}(x, y) = \frac{\sum_1^n (x_i - \bar{x})(y_i - \bar{y})}{n-1}$

μ = mean of population data.

$\bar{x}$ = mean of sample data.

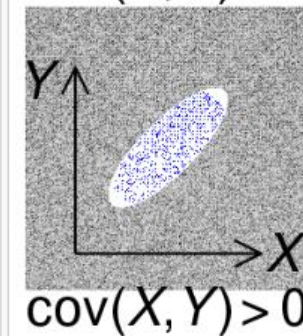
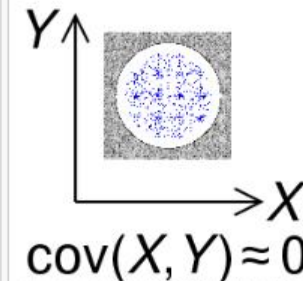
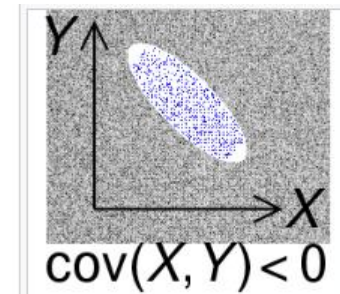
n = number of observations in the dataset.

x_i = observations in dataset x .

<https://towardsdatascience.com/why-gradient-descent-and-normal-equation-are-bad-for-linear-regression-928f8>

https://soohee410.github.io/least_squares_sol

<https://medium.com/@mathphy/determinant-equal-to-zero-error-404-not-at-all-c2bc24c4e43c>



The sign of the covariance of two random variables X and Y

다중 회귀 Perceptron with Normal Equation

- What is $X^T X$? => Covariance Matrix

$$\begin{bmatrix} \text{Var}(x_1) & \dots & \text{Cov}(x_n, x_1) \\ \vdots & \ddots & \vdots \\ \text{Cov}(x_n, x_1) & \dots & \text{Var}(x_n) \end{bmatrix}$$

Population Variance: $\text{var}(x) = \frac{\sum_1^n (x_i - \mu)^2}{n}$

Population Covariance: $\text{cov}(x, y) = \frac{\sum_1^n (x_i - \mu_x)(y_i - \mu_y)}{n}$

Sample Variance: $\text{var}(x) = \frac{\sum_1^n (x_i - \bar{x})^2}{n-1}$

Sample Covariance: $\text{cov}(x, y) = \frac{\sum_1^n (x_i - \bar{x})(y_i - \bar{y})}{n-1}$

μ = mean of population data.

$\bar{x}$ = mean of sample data.

n = number of observations in the dataset.

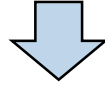
x_i = observations in dataset x .

What is $X^T X$? When $X = (X - \bar{X})$

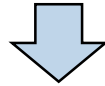
Do It!

다중 회귀 Perceptron 학습(파라미터 추정)

$$\text{minimize}_{\hat{w}} \text{Loss}(\hat{w}) = \text{minimize}_{\hat{w}} (Y - X\hat{w})^T (Y - X\hat{w})$$

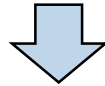


$$\hat{w} = (X^T X)^{-1} X^T Y \text{ is Unstable solution}$$



Inverse matrix of Covariance is Unstable or Covariance Not Invertible!

A square matrix is singular, that is, its determinant is zero, if it contains rows or columns which are proportionally interrelated; in other words, one or more of its rows (columns) is exactly expressible as a linear combination of all or some other its rows (columns), the combination being without a constant term.



It is when there is linear **interdependences** among the variables.

Linear Regression assumes that there is not linear **interdependences** among the variables

<https://towardsdatascience.com/why-gradient-descent-and-normal-equation-are-bad-for-linear-regression-928f8b32fa4f>

https://soohee410.github.io/least_squares_sol

<https://medium.com/@mathphy/determinant-equal-to-zero-error-404-not-at-all-c2bc24c4e43c>

<https://stats.stackexchange.com/questions/70899/what-correlation-makes-a-matrix-singular-and-what-are-implications-of-singularit>

다중 회귀 Perceptron with Normal Equation

So what does everyone use?

As we've seen the downsides of the approaches shown in ML courses, let's see what is used in practice. In Scikit-learn `LinearRegression` we can see:

Notes

From the implementation point of view, this is just plain Ordinary Least Squares (`scipy.linalg.lstsq`) wrapped as a predictor object.

So Scikit-learn does not bother with its own implementation, instead, it just uses Scipy. In `scipy.linalg.lstsq` we can see that even this library does not use its own implementation, instead using LAPACK:

`lapack_driver` : *str, optional*

Which LAPACK driver is used to solve the least-squares problem. Options are `'gelsd'`, `'gelsy'`, `'gelss'`. Default (`'gelsd'`) is a good choice. However, `'gelsy'` can be slightly faster on many problems. `'gelss'` was used historically. It is generally slow but uses less memory.

New in version 0.17.0.

Finally, we arrive at `gelsd`, `gelsy` and `gelss` entries in Intel LAPACK documentation:

다중 회귀 Perceptron with Normal Equation

`?ge ls`

Computes the minimum-norm solution to a linear least squares problem using the singular value decomposition of A and a divide and conquer method.

`?ge lsy`

Computes the minimum-norm solution to a linear least squares problem using a complete orthogonal factorization of A.

`?ge lss`

Computes the minimum-norm solution to a linear least squares problem using the singular value decomposition of A.

2 out of those 3 methods use Singular Value Decomposition (SVD), a very important algorithm both in numerical methods and machine learning. You may have heard about it in context of NLP or recommender systems, where it's used for dimensionality reduction. It turns out, it's also used for practical linear regression, where it provides a reasonably fast and very accurate method for computing least squares problem that lies in the heart of linear regression.

Moore-Penrose Inverse by using singular value decomposition

The inverse of a matrix A is another matrix A^{-1} that has this property:

$$\begin{aligned}AA^{-1} &= I \\ A^{-1}A &= I\end{aligned}$$

where I is the *identity matrix*. This is a nice property for a matrix to have, because then we can work with it in equations just like we might with ordinary numbers. For instance, to solve some linear system of equations

$$Ax = b$$

we can just multiply the inverse of A to both sides

$$x = A^{-1}b$$

and then we have some unique solution vector x . Again, this is just like we would do if we were trying to solve a real-number equation like $ax = b$.

Now, a matrix has an inverse whenever it is square and its rows are linearly independent. But not every system of equations we might care about will give us a matrix that satisfies these properties. The coefficient matrix A would fail to be invertible if the system did not have the same number of equations as unknowns (A is not square), or if the system had dependent equations (A has dependent rows).

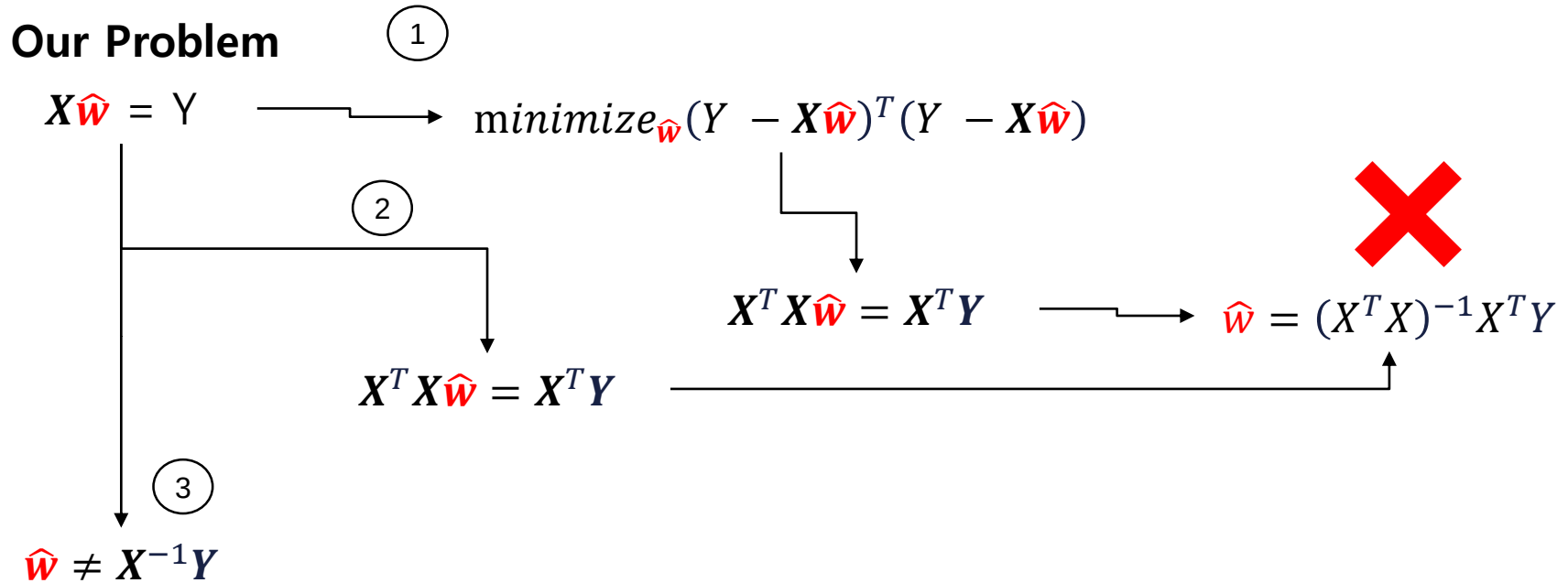
Generalized inverses are meant to solve this problem. They are meant to solve equations like $Ax = b$ in the “best way possible” when A^{-1} fails to exist. There are many kinds of generalized inverses, each with its own “best way.” (They can be used to solve ridge regression problems, for instance.)

The most common is the Moore-Penrose inverse, or sometimes just the pseudoinverse. It solves the least-squares problem for linear systems, and therefore will give us a solution $\hat{x}$ so that $A\hat{x}$ is as close as possible in ordinary Euclidean distance to the vector b .

The notation for the Moore-Penrose inverse is A^+ instead of A^{-1} . If A is invertible, then in fact $A^+ = A^{-1}$, and in that case the solution to the least-squares problem is the same as the ordinary solution ($A^+b = A^{-1}b$). So, the MP-inverse is strictly more general than the ordinary inverse: we can always use it and it will always give us the same solution as the ordinary inverse whenever the ordinary inverse exists.

We will look at how we can construct the Moore-Penrose inverse using the SVD. This turns out to be an easy extension to constructing the ordinary matrix inverse with the SVD. We will then see how solving a least-squares problem is just as easy as solving an ordinary equation.

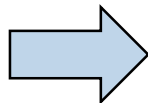
Moore-Penrose Inverse by using SVD



Exact Inverse

$$X^{-1}$$

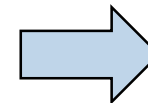
$$(X^T X)^{-1}$$



Moore-Penrose Inverse

$$X^+$$

$$(X^T X)^+$$

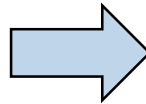


$$\hat{w} \approx X^+ Y$$

$$\hat{w} \approx (X^T X)^+ X^T Y$$

Moore-Penrose Inverse by using SVD

SVD



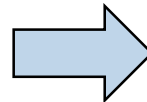
Moore-Penrose Inverse

$$\mathbf{X}^{-1} \approx \mathbf{X}^{+}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} \approx (\mathbf{X}^T \mathbf{X})^{+}$$

singular value decomposition

Any maxtrix $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ decompose



$$\mathbf{A}^{+} = \mathbf{U}\mathbf{\Sigma}^{+}\mathbf{V}^T$$

Moore-Penrose Invrse

Orthogonal Matrix:

- $\mathbf{U}\mathbf{U}^T = \mathbf{U}^T\mathbf{U} = \mathbf{I}$ or $\mathbf{V}\mathbf{V}^T = \mathbf{V}^T\mathbf{V} = \mathbf{I}$
- ith row vector $\perp$ jth row vector
- ith column vector $\perp$ jth column vector

$$\mathbf{U}^{-1} = \mathbf{U}^T$$

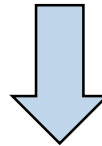
$$\mathbf{V}^{-1} = \mathbf{V}^T$$

SVD

Singular Value Decomposition

Any matrix $A = U\Sigma V^T$ decompose

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} V_{11} & V_{21} \\ V_{12} & V_{22} \end{bmatrix}$$



Moore-Penrose inverse By Using SVD

$$A^+ = U\Sigma^+V^T \quad \Rightarrow \quad = \begin{bmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{bmatrix} \begin{bmatrix} 1/\Sigma_1 & 0 \\ 0 & 1/\Sigma_2 \end{bmatrix} \begin{bmatrix} V_{11} & V_{21} \\ V_{12} & V_{22} \end{bmatrix}$$

Orthogonal Matrix:

- $UU^T = U^T U = I$ or $VV^T = V^T V = I$
- i th row vector $\perp$ j th row vector
- i th column vector $\perp$ j th column vector

$$U^{-1} = U^T$$

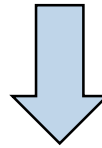
$$V^{-1} = V^T$$

Moore-Penrose Inverse by using SVD

Singular Value Decomposition

Any matrix $X = U\Sigma V^T$ decompose

$$\begin{bmatrix} X_{11} & X_{12} \\ X_{21} & X_{22} \end{bmatrix} = \begin{bmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} V_{11} & V_{21} \\ V_{12} & V_{22} \end{bmatrix}$$



Moore-Penrose inverse By Using SVD

$$X^+ = U\Sigma^+V^T \quad \Rightarrow \quad = \begin{bmatrix} U_{11} & U_{12} \\ U_{21} & U_{22} \end{bmatrix} \begin{bmatrix} 1/\Sigma_1 & 0 \\ 0 & 1/\Sigma_2 \end{bmatrix} \begin{bmatrix} V_{11} & V_{21} \\ V_{12} & V_{22} \end{bmatrix}$$

$$X^T X = V\Sigma^T U^T U \Sigma V^T = V\Sigma^2 V^T$$

$$(X^T X)^+ \approx V\Sigma^{*+}V^T$$

Orthogonal Matrix:

- $UU^T = U^T U = I$ or $VV^T = V^T V = I$
- i th row vector $\perp$ j th row vector
- i th column vector $\perp$ j th column vector

$$U^{-1} = U^T$$

$$V^{-1} = V^T$$

Moore-Penrose Inverse by using SVD

$$X^+ = U\Sigma^+V^T \quad \longrightarrow \quad \hat{w} \approx X^+Y \quad \longrightarrow \quad U\Sigma^+V^TY$$

$$(X^TX)^+ \approx V\Sigma^{*+}V^T \quad \longrightarrow \quad \hat{w} \approx (X^TX)^+X^TY \quad \longrightarrow \quad V\Sigma^{*+}V^T(V\Sigma^TU^T)Y$$

$$V\Sigma^{*+}\Sigma^TU^TY$$

$$\Sigma^{*+} \quad \Sigma^T$$

$$\begin{bmatrix} 1/\Sigma_1^2 & 0 \\ 0 & 1/\Sigma_2^2 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} = \begin{bmatrix} 1/\Sigma_1 & 0 \\ 0 & 1/\Sigma_2 \end{bmatrix} = \Sigma^+$$

Moore-Penrose Inverse by using SVD

$$X^+ = U\Sigma^+V^T \quad \longrightarrow \quad \hat{w} \approx X^+Y \quad \longrightarrow \quad U\Sigma^+V^TY$$

$$(X^TX)^+ \approx V\Sigma^{*+}V^T \quad \longrightarrow \quad \hat{w} \approx (X^TX)^+X^TY \quad \longrightarrow \quad V\Sigma^{*+}V^T(V\Sigma^TU^T)Y$$

$$V\Sigma^{*+}\Sigma^TU^TY$$

$$\Sigma^{*+} \quad \Sigma^T$$

$$\begin{bmatrix} 1/\Sigma_1^2 & 0 \\ 0 & 1/\Sigma_2^2 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} = \begin{bmatrix} 1/\Sigma_1 & 0 \\ 0 & 1/\Sigma_2 \end{bmatrix} = \Sigma^+$$

$\hat{w} \approx X^+Y = (X^TX)^+X^TY$

살펴보기 - 사이트

In Ordinary Least Squares (OLS) Linear Regression, our goal is to find the line (or hyperplane) that minimizes the vertical offsets. Or in other words, we define the best-fitting line as the line that minimizes the sum of squared errors (SSE) or mean squared error (MSE) between our target variable (y) and our predicted output over all samples i in our dataset of size n .

$$SSE = \sum_i (\text{target}^{(i)} - \text{output}^{(i)})^2$$

$$MSE = \frac{1}{n} \times SSE$$

Now, `LinearRegression` implements a linear regression model for performing ordinary least squares regression using one of the following five approaches:

- Normal Equations
- QR Decomposition Method
- SVD (Singular Value Decomposition) method
- Gradient Descent
- Stochastic Gradient Descent

Normal Equations (closed-form solution)

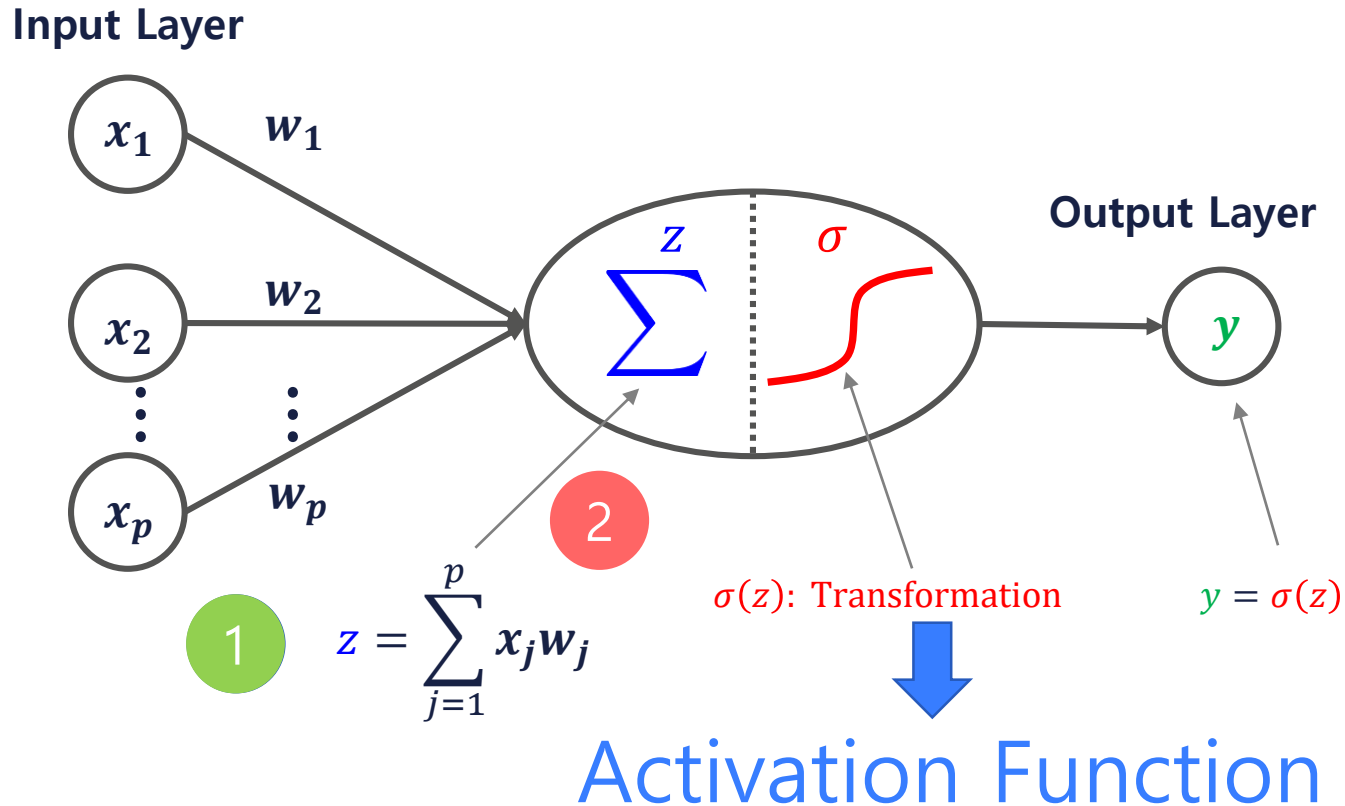
The closed-form solution should be preferred for "smaller" datasets where calculating (a "costly") matrix inverse is not a concern. For very large datasets, or datasets where the inverse of $[X^T X]$ may not exist (the matrix is non-invertible or singular, e.g., in case of perfect multicollinearity), the QR, SVD or gradient descent approaches are to be preferred.

The linear function (linear regression model) is defined as:

$$y = w_0 x_0 + w_1 x_1 + \dots + w_m x_m = \sum_{i=0}^m = \mathbf{w}^T \mathbf{x}$$

https://rasbt.github.io/mlxtend/user_guide/regressor/LinearRegression/

Linear and Logistic Regression & ANN (Perceptron)



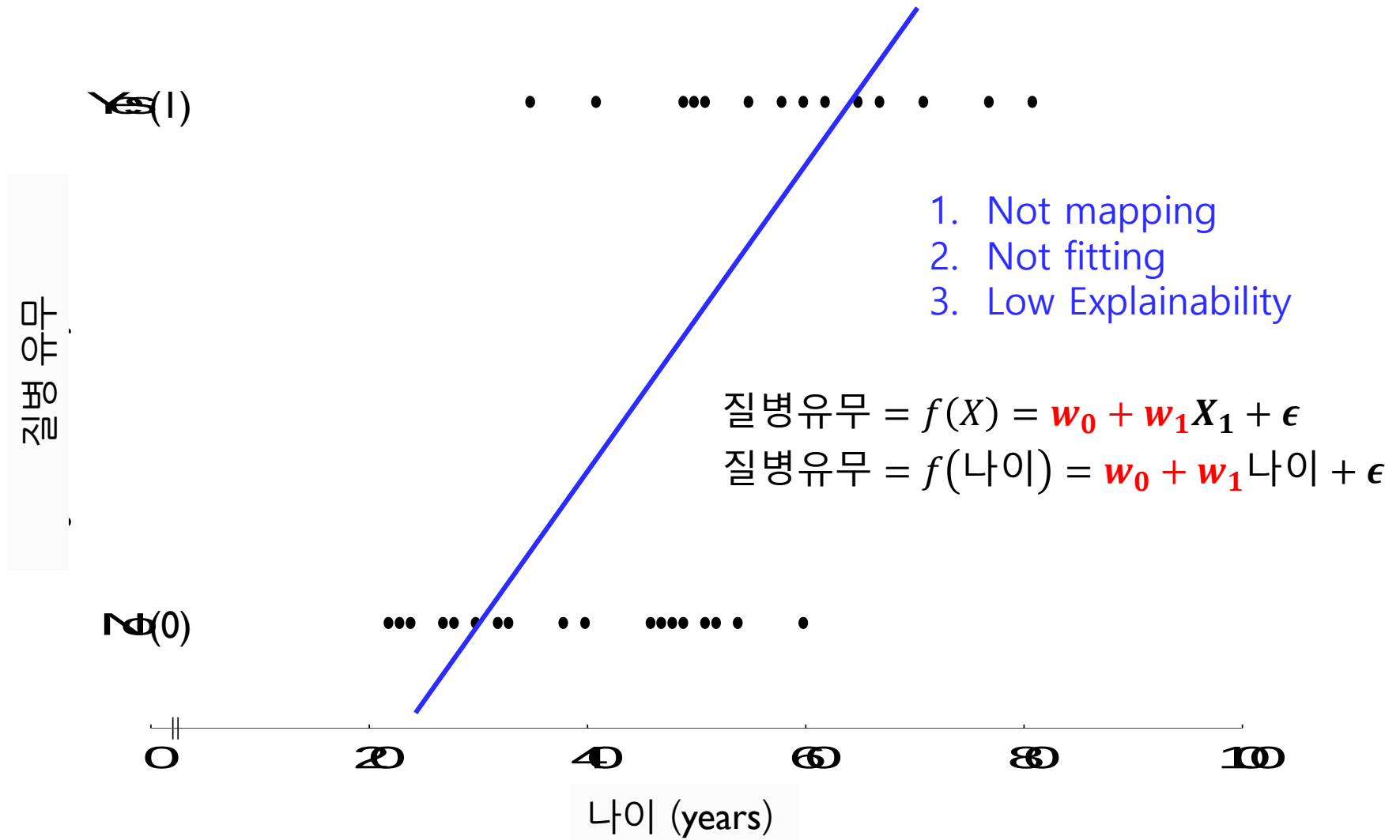
1 입력변수의 선형결합

2 선형결합 값의 변환 (Linear or Nonlinear Transformation)

Linear and Logistic Regression & ANN (Perceptron)

나이	질병유무	나이	질병유무	나이	질병유무
22	0	40	0	54	0
23	0	41	1	55	1
24	0	46	0	58	1
27	0	47	0	60	1
28	0	48	0	60	0
30	0	49	1	62	1
30	0	49	0	65	1
32	0	50	1	67	1
33	0	51	0	71	1
35	1	51	1	77	1
38	0	52	0	81	1

Linear and Logistic Regression & ANN (Perceptron)



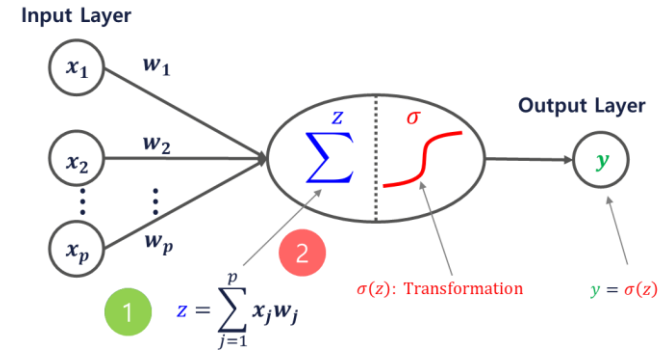
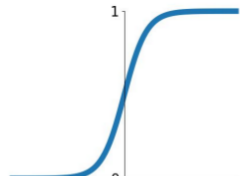
Logistic Regression & ANN (Perceptron)-확률 개념

Activation Functions (활성함수)

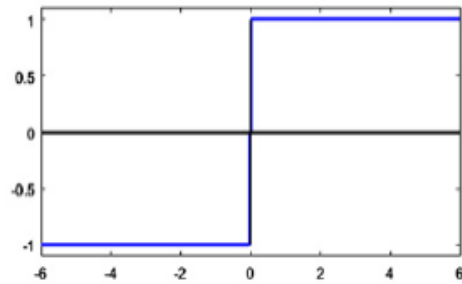
Nonlinear Transformation

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Step

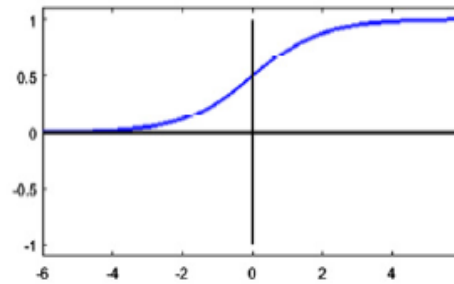


(a) 계단 함수

<

Sigmoid

Why? 다음 장

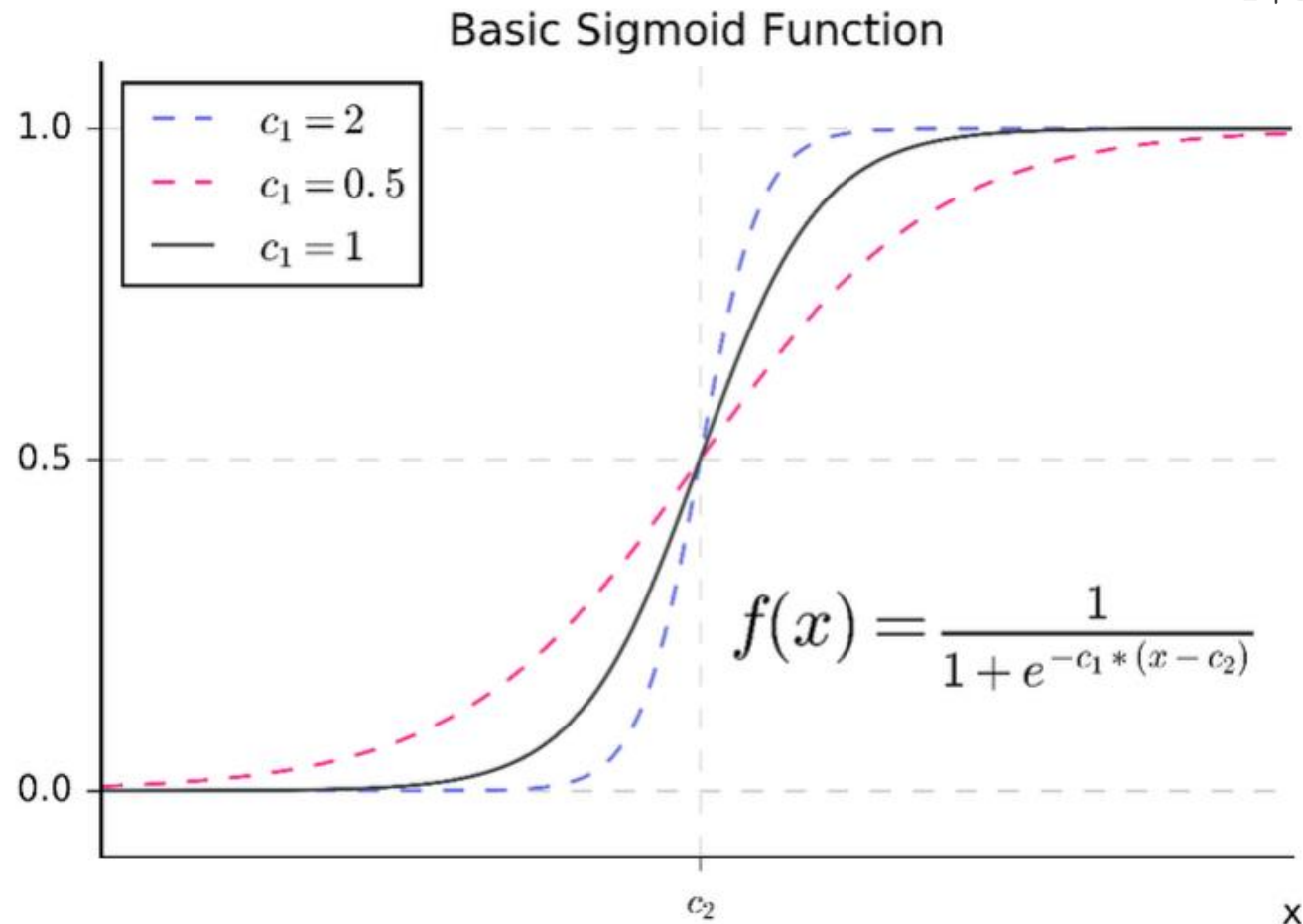
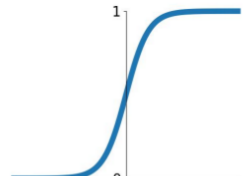


(b) 로지스틱 시그모이드

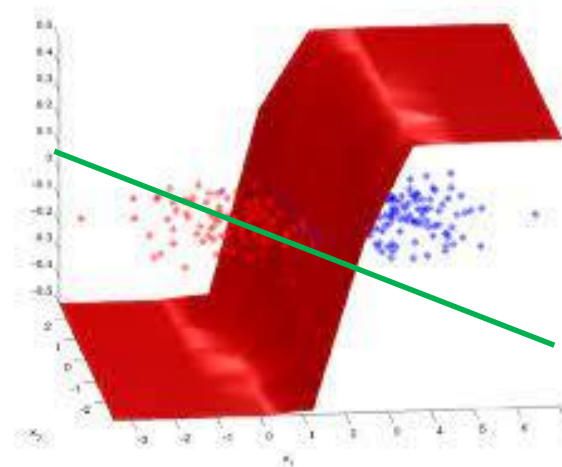
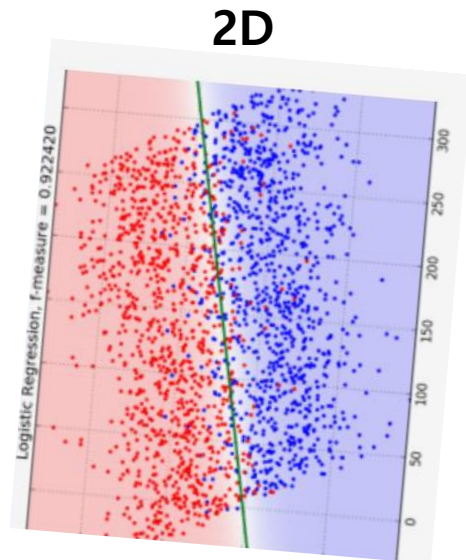
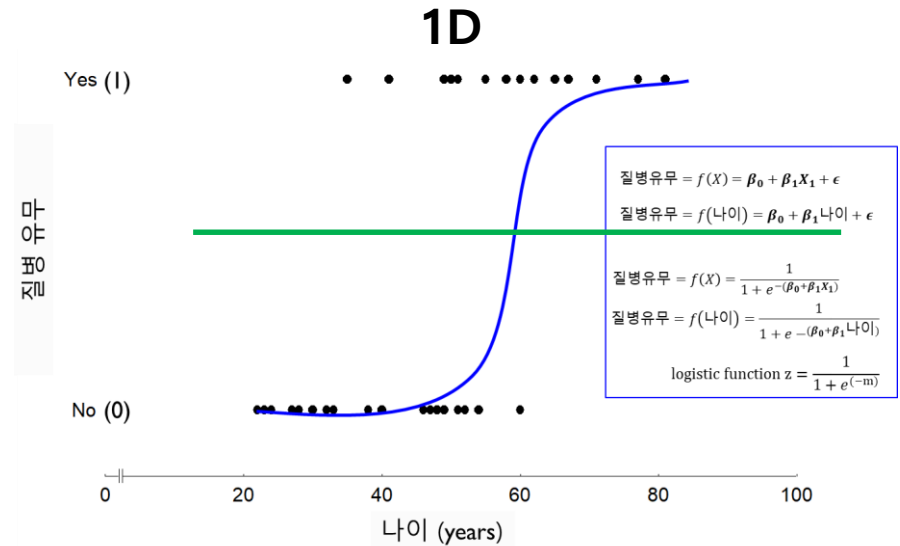
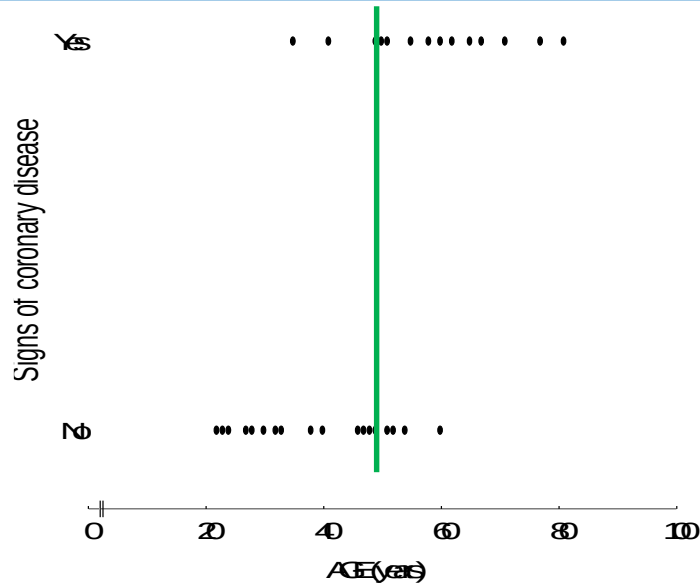
Logistic or Sigmoid Function

Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

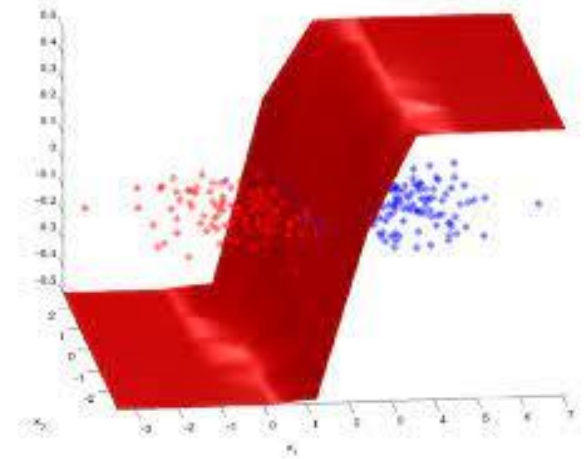
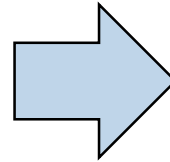
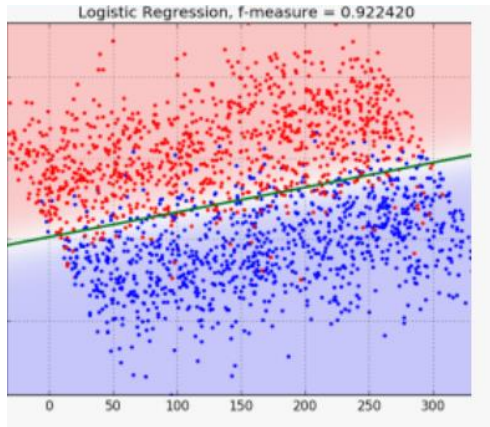


Decision Boundaries

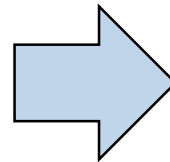
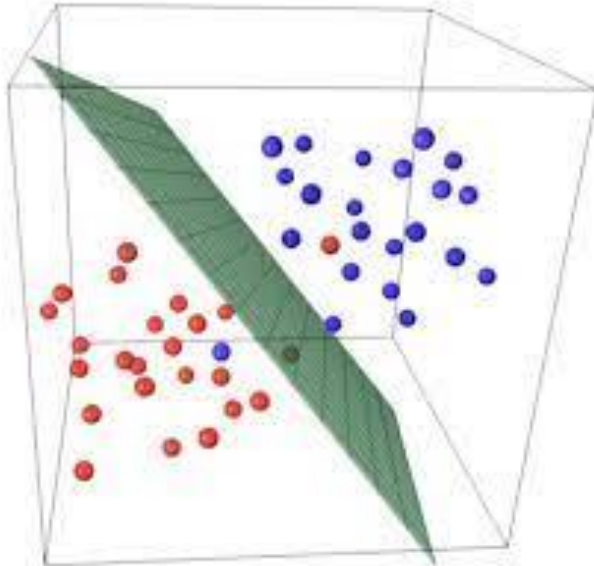


Decision Boundaries

2D



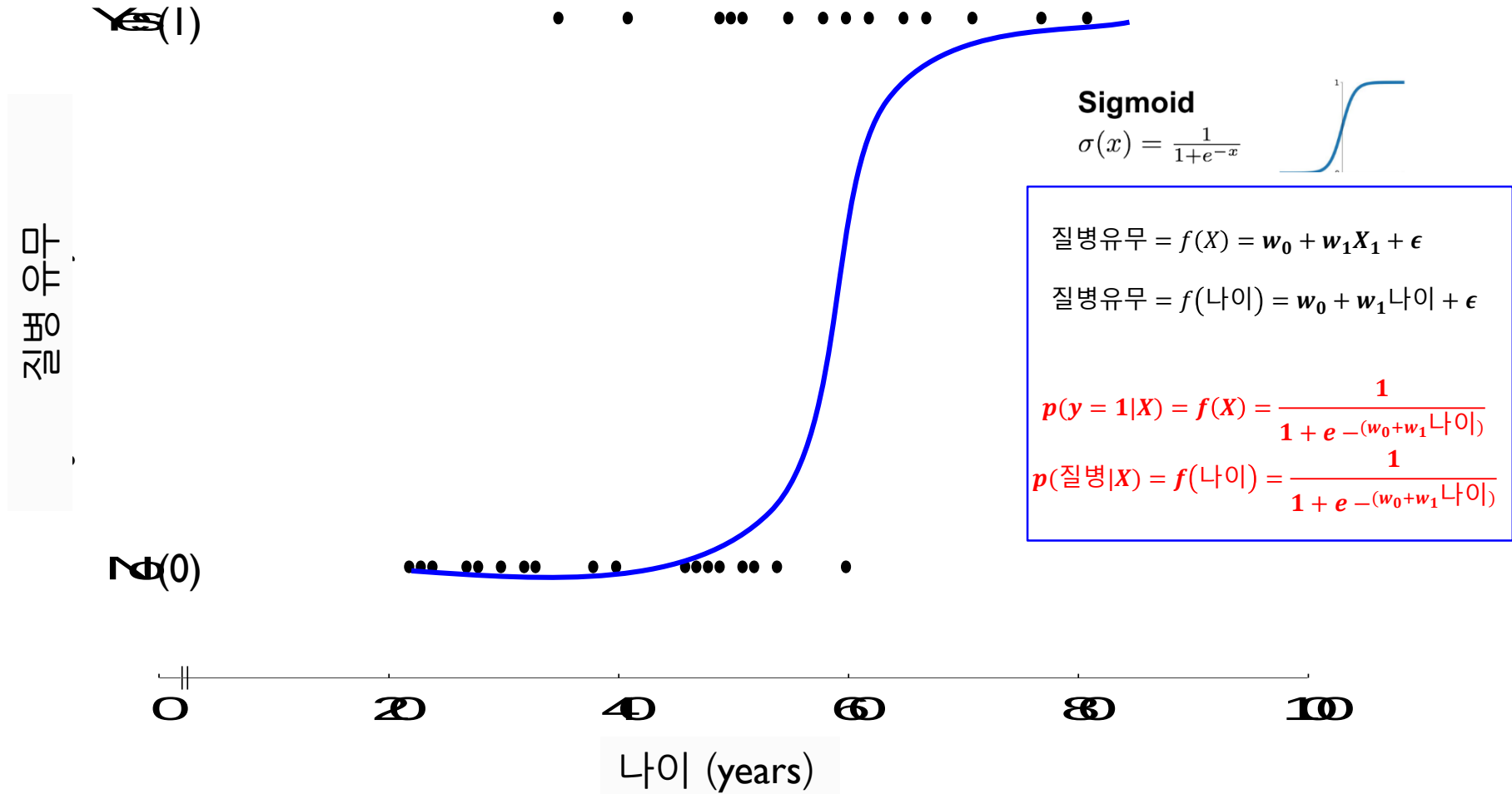
3D



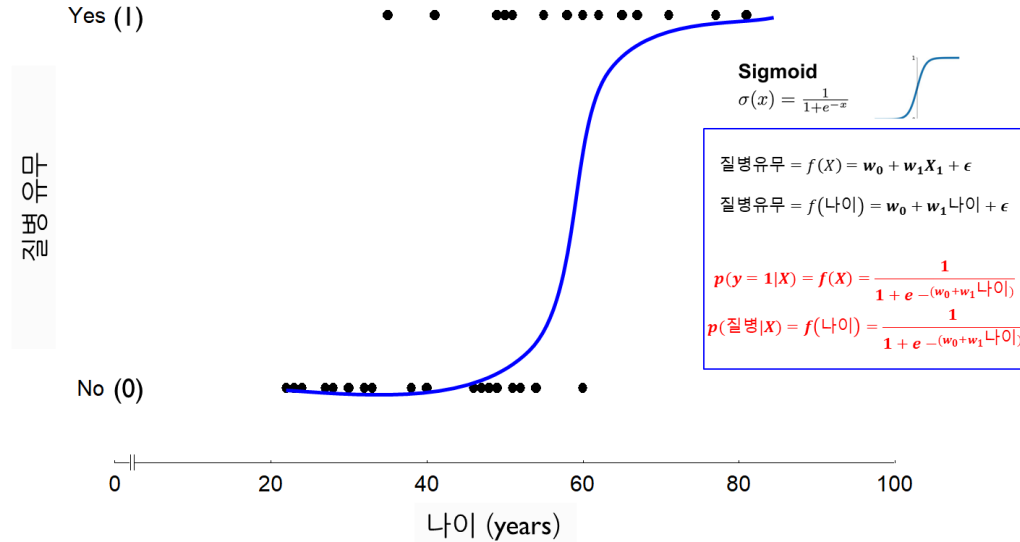
Hyperplane > 3D

Logistic Regression & ANN (Perceptron)

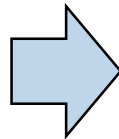
어떤 차이가 있는지 추측!



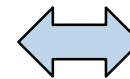
Perceptron- 확률 개념



나이 x	y
28	정상
32	정상
46	암
52	암



나이 x	Y P(암)
28	0
32	0
46	1
52	1



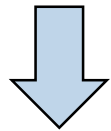
나이 x	$\hat{y}$
28	0.3
32	0.6
46	0.8
52	1.

Loss Function in Classification

$$p(\text{질병}|X) = f(\text{나이}) = \frac{1}{1 + e^{-(w_0 + w_1 \text{나이})}} \quad \Rightarrow \quad \hat{f}(\text{나이}) = \frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 \text{나이})}}$$

why not ? Squared Error Loss

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\})^2$$



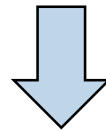
Try!!!

Loss Function in Classification

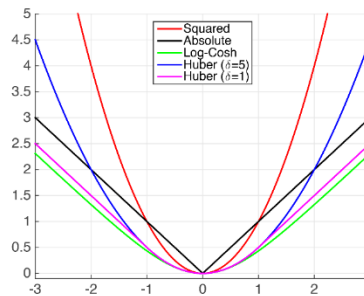
$$p(\text{질병}|X) = f(\text{나이}) = \frac{1}{1 + e^{-(w_0 + w_1 \text{나이})}} \quad \Rightarrow \quad \hat{f}(\text{나이}) = \frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 \text{나이})}}$$

why not ? Squared Error Loss

$$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\})^2$$



$$\frac{1}{n} \sum_{i=1}^n \left(y_i - \left\{ \frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right\} \right)^2$$



나이 x	Y P(질)	나이 x	$\hat{y}$
28	0	28	0.3
32	0	32	0.6
46	1	46	0.8
52	1	52	1.

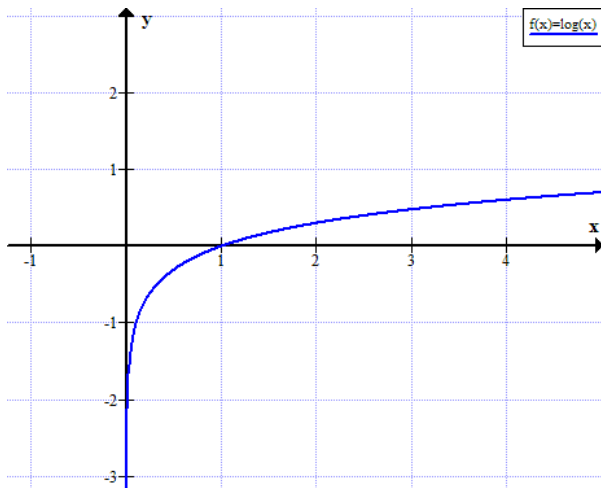
$$(0 - \mathbf{0.6})^2 \quad (0 - \mathbf{1})^2$$

Loss Function in Classification

why not ? why not ? Squared Error Loss

$$\text{Squared Error Loss} = (y - \hat{y})^2 = (1 - 0)^2$$

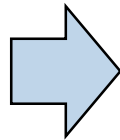
$$\text{binary cross entropy} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$



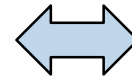
계산해보기~

Loss Function in Classification

나이 x	y
28	정상
32	정상
46	암
52	암

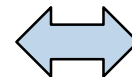


나이 x	$Y_{P(\text{암})}$
28	0
32	0
46	1
52	1



나이 x	$\hat{y}$
28	0.3
32	0.6
46	0.8
52	1.

나이 x	$Y_{P(\text{암})}$	$1-Y_{P(\text{정상})}$
28	0	1
32	0	1
46	1	0
52	1	0



나이 x	$\hat{y}_{P(\text{암})}$	$1-\hat{y}_{P(\text{정상})}$
28	0.3	0.7
32	0.6	0.4
46	0.8	0.2
52	1.	0.

Loss Function in Classification

why not ? why not ? Squared Error Loss

$$\text{squared Error Loss} = (y - \hat{y})^2 = (1 - 0)^2$$

$$\text{binary cross entropy} = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

실제와예측 분포간의 유사성

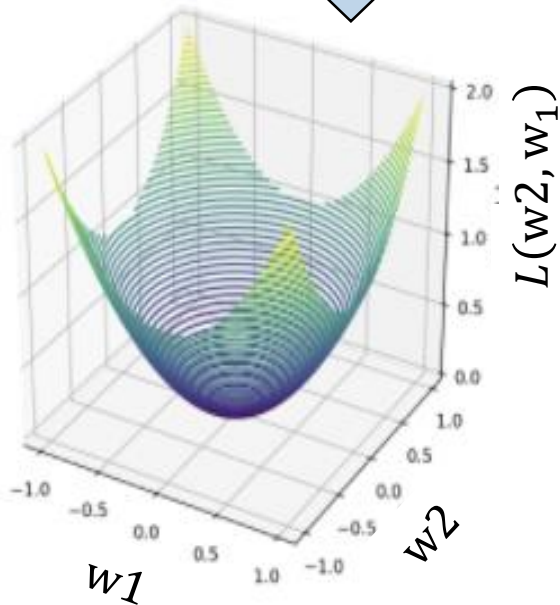
나이 x	Y P(암)	1-Y P(정상)	나이 x	$\hat{y}$ P(암)	1- $\hat{y}$ P(정상)
28	0	1	28	0.3	0.7
32	0	1	32	0.6	0.4
46	1	0	46	0.8	0.2
52	1	0	52	1.	0.

Optimization & Loss Function in Classification

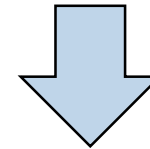
$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \hat{y}_i)^2$$



$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \{\hat{w}_0 + \hat{w}_1 x_i\})^2$$



$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m - \left\{ y_i \log \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right) + (1 - y_i) \log \left(1 - \frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right) \right\}$$



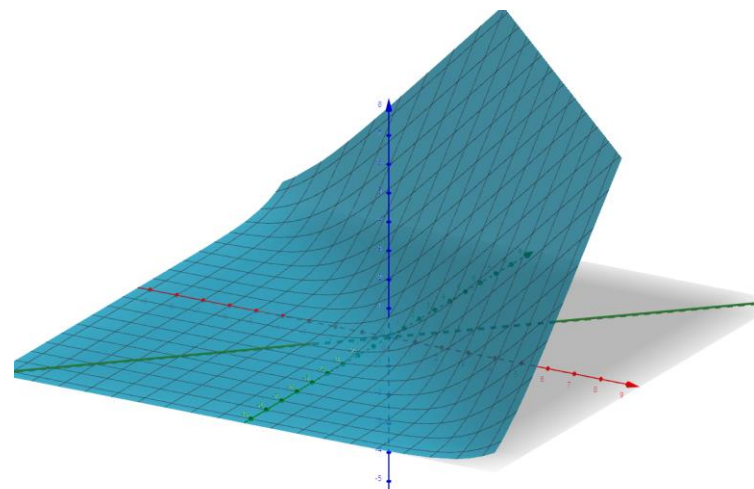
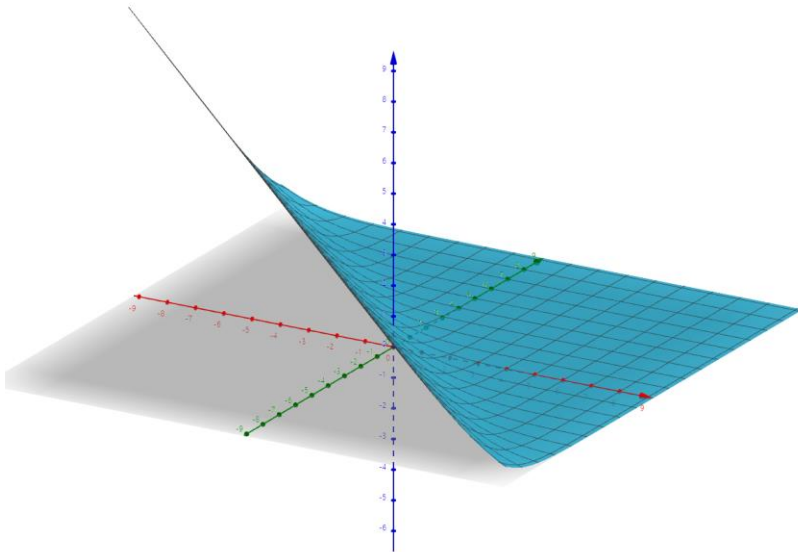
Optimization & Loss Function in Classification

$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{i=1}^m - \left\{ y_i \log \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_{i1})}} \right) + (1 - y_i) \log \left(1 - \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_{i1})}} \right) \right) \right\}$$

$$\left\{ y_i \log \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_{i1})}} \right) + (1 - y_i) \log \left(1 - \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_{i1})}} \right) \right) \right\}$$

$$y_i = 1$$

$$y_i = 0$$



$$x_1 = 1$$

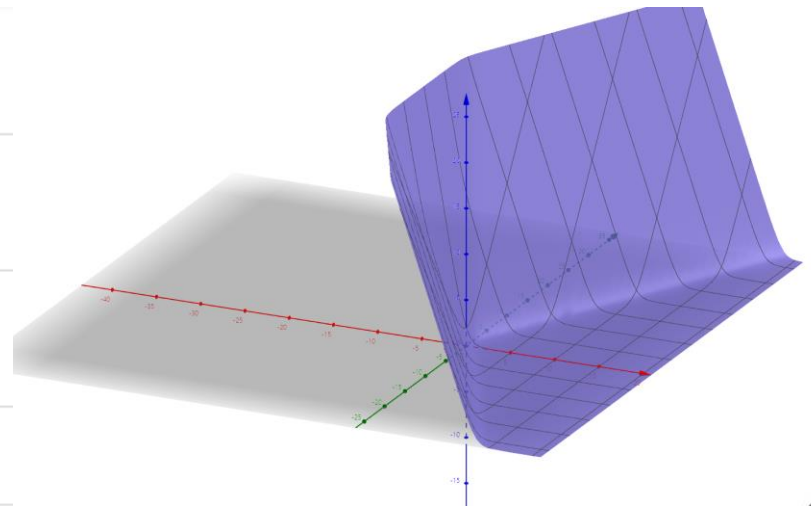
Optimization & Loss Function in Classification

<https://www.geogebra.org/3d>

$$(x_1 = 2, y_1 = 1)$$

$$(x_1 = -2, y_1 = 0)$$

☐	$a : z = -\log_2\left(\frac{1}{1 + e^{-(2x+y)}}\right)$
☐	$b(x, y) = -\log_2\left(1 - \frac{1}{1 + e^{2x-y}}\right)$
☐	$c(x, y) = -\log_2\left(\frac{1}{1 + e^{-(2x+y)}}\right) - \log_2\left(1 - \frac{1}{1 + e^{2x-y}}\right)$
☐	입력...



Optimization & Loss Function in Classification

$$\text{minimize}_{w_0, w_1} \sum_{i=1}^m - \left\{ y_i \log \left(\frac{1}{1 + e^{-(w_0 + w_1 x_i)}} \right) + (1 - y_i) \log \left(1 - \left(\frac{1}{1 + e^{-(w_0 + w_1 x_i)}} \right) \right) \right\}$$

$$(x_1 = 2, y_1 = 1)$$

$$(x_1 = -2, y_1 = 0)$$



$$L(w_0, w_1) = -\log \left(\frac{1}{1 + e^{-(w_0 + 2w_1)}} \right) - \log \left(1 - \frac{1}{1 + e^{-(w_0 - 2w_1)}} \right)$$



$$L(w_0, w_1) = -q(\sigma(\text{net}(w|x = 2))) - q(1 - \sigma(\text{net}(w|x = -2)))$$

$$m = q(k) = \log(k)$$

$$\frac{\partial m}{\partial k} = \frac{\partial q(k)}{\partial k} = \frac{1}{k}$$

$$k = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial k}{\partial z} = \frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

$$z = \text{net}(w|x) = w_0 + w_1 x$$

$$\frac{\partial z}{\partial w_0} = \frac{\partial \text{net}(w|x)}{\partial w_0} = 1$$

$$\frac{\partial z}{\partial w_1} = x$$

$$\frac{\partial L(w_0, w_1)}{\partial w_0} = - \frac{\partial q(\sigma(\text{net}(w|x = 2)))}{\partial w_0} - \frac{\partial q(1 - \sigma(\text{net}(w|x = -2)))}{\partial w_0}$$

$$= - \frac{1}{k_1} \sigma(z_1)(1 - \sigma(z_1)) + \frac{1}{k_2} \sigma(z_2)(1 - \sigma(z_2))$$

$$= - \frac{1}{\sigma(z_1)} \sigma(z_1)(1 - \sigma(z_1)) + \frac{1}{(1 - \sigma(z_2))} \sigma(z_2)(1 - \sigma(z_2))$$

$$= -(1 - \sigma(z_1)) + \sigma(z_2)$$

$$= - \left(1 - \frac{1}{1 + e^{-(w_0 + w_1 2)}} \right) + \frac{1}{1 + e^{-(w_0 - w_1 2)}}$$

$$= - \frac{e^{-(w_0 + w_1 2)}}{1 + e^{-(w_0 + w_1 2)}} + \frac{1}{1 + e^{-(w_0 - w_1 2)}}$$

Optimization & Loss Function in Classification

$$\frac{\partial L(w_0, w_1)}{\partial w_0} = -\frac{e^{-(w_0+w_1x)}}{1 + e^{-(w_0+w_1x)}} + \frac{1}{1 + e^{-(w_0-w_1x)}}$$

$$\frac{\partial L(w_0, w_1)}{\partial w_1} = -\frac{\partial q(\sigma(\text{net}(w|x = 2)))}{\partial w_1} - \frac{\partial q(1 - \sigma(\text{net}(w|x = -2)))}{\partial w_1}$$

$$= -\frac{1}{k_1} \sigma(z_1) (1 - \sigma(z_1))2 + \frac{1}{k_2} \sigma(z_2) (1 - \sigma(z_2))(-2)$$

$$m = q(k) = \log(k)$$

$$\frac{\partial m}{\partial k} = \frac{\partial q(k)}{\partial k} = \frac{1}{k}$$

$$k = \sigma(z) = \frac{1}{1 + e^{-(z)}}$$

$$\frac{\partial k}{\partial z} = \frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

$$z = \text{net}(w|x) = w_0 + w_1x$$

$$\frac{\partial z}{\partial w_0} = \frac{\partial \text{net}(w|x)}{\partial w_0} = 1$$

$$\frac{\partial z}{\partial w_1} = x$$

$$= -\frac{1}{\sigma(z_1)} \sigma(z_1) (1 - \sigma(z_1))2 + \frac{1}{(1 - \sigma(z_2))} \sigma(z_2) (1 - \sigma(z_2))(-2)$$

$$= -2(1 - \sigma(z_1)) - 2\sigma(z_2)$$

$$= -2\left(1 - \frac{1}{1 + e^{-(w_0+w_1x)}}\right) - \frac{2}{1 + e^{-(w_0-w_1x)}}$$

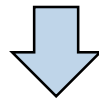
Optimization & Loss Function in Classification

For regression,

$$\hat{y} = \hat{f}(x) = \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3 + \hat{w}_0$$

$$Loss(\hat{\mathbf{w}}|X, Y) = \sum_i^n (y^i - \hat{y}^i)^2$$

$$\text{minimize}_{\hat{\mathbf{w}}} (Y - X\hat{\mathbf{w}})^T (Y - X\hat{\mathbf{w}})$$



$$X^T X \hat{\mathbf{w}} = X^T Y$$

- Normal Equation => Closed form solution
- Approximation by svd => Numerical solution

$$\hat{\mathbf{w}} = \text{or } \approx (X^T X)^{-1} X^T Y$$

For classification,

$$Loss(\hat{\mathbf{w}}|X, Y) = - \sum_i^n y^i \log \hat{y}^i + (1 - y^i) \log(1 - \hat{y}^i)$$

$$p(y = 1) = \hat{f}(x) = \frac{1}{1 + e^{-(\hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3 + \hat{w}_0)}}$$

$$\text{minimize}_{\hat{\mathbf{w}}} - \sum_i^n y^i \log \hat{y}^i + (1 - y^i) \log(1 - \hat{y}^i)$$

$$\text{minimize}_{\hat{\mathbf{w}}} - Y^T \log \sigma(X\hat{\mathbf{w}}) - (1 - Y^T) \log \sigma(X\hat{\mathbf{w}})$$

Optimization & Loss Function in Classification

For regression,

$$\hat{y} = \hat{f}(x) = \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3 + \hat{w}_0$$

$$\text{minimize}_{\hat{w}} (Y - X\hat{w})^T (Y - X\hat{w})$$

$$\frac{\partial \text{Loss}(\hat{w})}{\partial \hat{w}}$$



$$X^T X \hat{w} = X^T Y$$

$$\hat{w} = \text{or } \approx (X^T X)^{-1} X^T Y$$

For classification,

$$p(y = 1) = \hat{f}(x) = \frac{1}{1 + e^{-(\hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3 + \hat{w}_0)}}$$

$$\text{minimize}_{\hat{w}} -Y^T \log \sigma(X\hat{w}) - (1 - Y^T) \log \sigma(X\hat{w})$$

$$\frac{\partial \text{Loss}(\hat{w})}{\partial \hat{w}}$$



- Normal Equation => Closed form solution
- Approximation by svd => Numerical solution

$$X^T \log \sigma(X\hat{w}) = X^T Y$$



Gradient Descent (경사하강법) in Logistic Regression

$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m - \left\{ y_i \log \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right) + (1 - y_i) \log \left(1 - \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right) \right) \right\}$$

First-order (Gradient Decent)
optimization methods 경사하강법

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \frac{\partial \text{Loss}(\hat{\mathbf{w}}(k))}{\partial \hat{\mathbf{w}}(k)}$$

$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \begin{bmatrix} \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_0} \\ \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_1} \end{bmatrix}$$



Gradient Descent (경사하강법) in Logistic Regression

$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m - \left\{ y_i \log \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right) + (1 - y_i) \log \left(1 - \left(\frac{1}{1 + e^{-(\hat{w}_0 + \hat{w}_1 x_i)}} \right) \right) \right\}$$

First-order (Gradient Decent)
optimization methods 경사하강법

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \frac{\partial \text{Loss}(\hat{\mathbf{w}}(k))}{\partial \hat{\mathbf{w}}(k)}$$

합성함수 형태

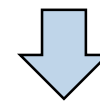
$$\frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} = \begin{bmatrix} \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_0} \\ \frac{\partial \text{Loss}(\hat{\mathbf{w}})}{\partial \hat{w}_1} \end{bmatrix}$$



$$l_i = L(\hat{y}_i) = -\{y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\}$$

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-(z_i)}}$$

$$z_i = \text{net}(w|x_i) = w_0 + w_1 x_{i1}$$



$$l_i = L(\hat{y}_i) = L(\sigma(z_i)) = L(\sigma(\text{net}(w)))$$

합성함수

$$l_i = L(\hat{y}_i) = L(\sigma(z_i)) = L(\sigma(\text{net}(w|x_{i1})))$$

$$= \frac{\partial l_i}{\partial w_1} = \frac{\partial L(\sigma(\text{net}(w|x_{i1})))}{\partial w_1}$$

Chain Rule

$$\frac{\partial l_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_i} \frac{\partial z_i}{\partial \hat{w}_1}$$

$$l_i = L(\hat{y}_i) = -\{y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\}$$

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-(z_i)}}$$

$$z_i = \text{net}(w) = w_0 + w_1 x_{i1}$$

미분 생각해보기!!!!

합성함수

$$l_i = L(\hat{y}_i) = L(\sigma(z_i)) = L(\sigma(\text{net}(w|x_{i1})))$$

$$= \frac{\partial l_i}{\partial w_1} = \frac{\partial L(\sigma(\text{net}(w|x_{i1})))}{\partial w_1}$$

Chain Rule

$$\frac{\partial l_i}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial z_i} \frac{\partial z_i}{\partial \hat{w}_1}$$

$$l_i = L(\hat{y}_i) = -\{y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\} \quad \frac{\partial l_i}{\partial \hat{y}_i} = -\left\{y_i \frac{1}{\hat{y}_i} - (1 - y_i) \frac{1}{(1 - \hat{y}_i)}\right\}$$

$$\hat{y}_i = \sigma(z_i) = \frac{1}{1 + e^{-(z_i)}}$$

$$\frac{\partial \hat{y}_i}{\partial z_i} = \sigma(z_i) \{1 - \sigma(z_i)\}$$

$$z_i = \text{net}(w) = w_0 + w_1 x_{i1}$$

$$\frac{\partial z_i}{\partial \hat{w}_1} = x_{i1}$$

합성함수

Chain Rule

$$\frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_i} \frac{dz_i}{d\hat{w}_1}$$

$$\frac{\partial l_i}{\partial w_1} = - \left\{ y_i \frac{1}{\hat{y}_i} - (1 - y_i) \frac{1}{(1 - \hat{y}_i)} \right\} \sigma(z_i) \{1 - \sigma(z_i)\} x_{i1}$$

$$\frac{\partial l_i}{\partial w_1} = - \left\{ y_i \frac{1}{\sigma(z_i)} - (1 - y_i) \frac{1}{(1 - \sigma(z_i))} \right\} \sigma(z_i) \{1 - \sigma(z_i)\} x_{i1}$$

$$\frac{\partial l_i}{\partial w_1} = - \left\{ \frac{(1 - \sigma(z_i)) y_i - (1 - y_i) \sigma(z_i)}{\sigma(z_i) (1 - \sigma(z_i))} \right\} \sigma(z_i) \{1 - \sigma(z_i)\} x_{i1} = - \left\{ \frac{y_i - \sigma(z_i)}{\sigma(z_i) (1 - \sigma(z_i))} \right\} \sigma(z_i) \{1 - \sigma(z_i)\} x_{i1}$$

$$\frac{\partial l_i}{\partial \hat{y}_i} = \{ y_i \log \hat{y}_i - (1 - y_i) \log (1 - \hat{y}_i) \}$$

$$\frac{\partial \hat{y}_i}{\partial z_i} = \sigma(z_i) \{1 - \sigma(z_i)\}$$

$$\frac{\partial z_i}{\partial w_1} = x_{i1}$$

Perceptron - 경사하강법

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \frac{\partial \text{Loss}(\hat{\mathbf{w}}(k))}{\partial \hat{\mathbf{w}}(k)}$$

$$\frac{\partial l_i}{\partial \mathbf{w}_1} = - \left\{ \frac{(1-\sigma(\mathbf{z}_i))y_i - (1-y_i)\sigma(\mathbf{z}_i)}{\sigma(\mathbf{z}_i)(1-\sigma(\mathbf{z}_i))} \right\} \sigma(\mathbf{z}_i) \{1 - \sigma(\mathbf{z}_i)\} x_{i1} = - \left\{ \frac{y_i - \sigma(\mathbf{z}_i)}{\sigma(\mathbf{z}_i)(1-\sigma(\mathbf{z}_i))} \right\} \sigma(\mathbf{z}_i) \{1 - \sigma(\mathbf{z}_i)\} x_{i1}$$

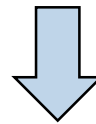
$$\frac{\partial l_i}{\partial \mathbf{w}_1} = - \{y_i - \sigma(\mathbf{z}_i)\} x_{i1}$$

$$\mathbf{w}_1(k+1) := \mathbf{w}_1(k) - \alpha \cdot -\{y_i - \sigma(\mathbf{z}_i)\} x_{i1}$$

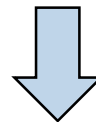
$$\mathbf{w}_1(k+1) := \mathbf{w}_1(k) + \alpha \cdot \{y_i - \hat{y}_i\} x_{i1}$$

Perceptron - 합성함수

$$w_1(k+1) := w_1(k) - \alpha \cdot -\{y_i - \sigma(z_i)\}x_{i1}$$



$$w_1(k+1) := w_1(k) + \alpha \cdot \{y_i - \hat{y}_i\}x_{i1}$$



$$w_1(k+1) := w_1(k) + \alpha \cdot \sum_{i=1}^m \{y_i - \hat{y}_i\}x_{i1}$$

Solver in Logistic Regression

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

`solver : {'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'}, default='lbfgs'`

Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- For multiclass problems, only 'newton-cg', 'sag', 'saga' and 'lbfgs' handle multinomial loss;
- 'liblinear' is limited to one-versus-rest schemes.

Solver in Logistic Regression

Numerical Solution

From the perspective of the gradient information in optimization, popular optimization methods can be divided into **three categories**:

1. First-order (Gradient Decent) optimization methods 경사하강법

$$w_{k+1} = w_k - \alpha \cdot \text{Loss}^{(1)}(w_k)$$

2. High-order optimization methods (ex: Newton's method)

$$w_{k+1} = w_k - \frac{\text{Loss}^{(1)}(x_k)}{\text{Loss}^{(2)}(x_k)}$$

3. Heuristic derivative-free optimization methods

Solver in Logistic Regression

Trick: Taylor Expansion

How can you minimize a function ℓ if you don't know much about it? The trick is to assume it is much simpler than it really is. This can be done with Taylor's approximation. Provided that the norm $\|\mathbf{s}\|_2$ is small (*i.e.* $\mathbf{w} + \mathbf{s}$ is very close to $\mathbf{w}$), we can approximate the function $\ell(\mathbf{w} + \mathbf{s})$ by its first and second derivatives:

$$\ell(\mathbf{w} + \mathbf{s}) \approx \ell(\mathbf{w}) + g(\mathbf{w})^\top \mathbf{s}$$

$$\ell(\mathbf{w} + \mathbf{s}) \approx \ell(\mathbf{w}) + g(\mathbf{w})^\top \mathbf{s} + \frac{1}{2} \mathbf{s}^\top H(\mathbf{w}) \mathbf{s}$$

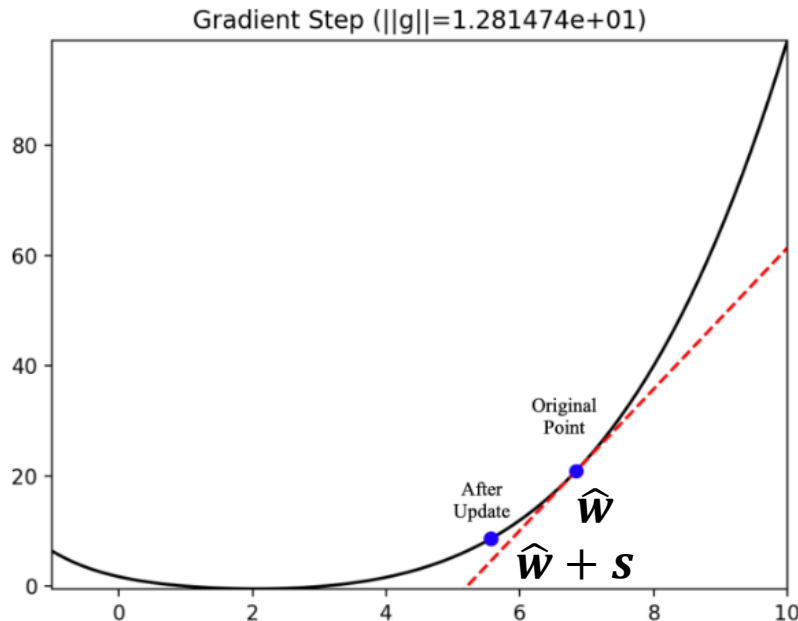
Here, $g(\mathbf{w}) = \nabla \ell(\mathbf{w})$ is the gradient and $H(\mathbf{w}) = \nabla^2 \ell(\mathbf{w})$ is the Hessian of ℓ . Both approximations are valid if $\|\mathbf{s}\|_2$ is small, but the second one assumes that ℓ is twice differentiable and is more expensive to compute but also more accurate than only using gradient.

$$Loss(\hat{\mathbf{w}} + \mathbf{s}) \approx Loss(\hat{\mathbf{w}}) + \left(\frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} \right)^\top \mathbf{s}$$

$$Loss(\hat{\mathbf{w}} + \mathbf{s}) \approx Loss(\hat{\mathbf{w}}) + \left(\frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} \right)^\top \mathbf{s} + \frac{1}{2} \mathbf{s}^\top H(\hat{\mathbf{w}}) \mathbf{s}$$

Solver in Logistic Regression

$$Loss(\hat{\mathbf{w}} + \mathbf{s}) \approx Loss(\hat{\mathbf{w}}) + \left(\frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} \right)^T \mathbf{s}$$



$$Loss(\mathbf{x}) \approx \frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} (\mathbf{x} - \hat{\mathbf{w}}) + Loss(\hat{\mathbf{w}})$$



$$Loss(\hat{\mathbf{w}} + \mathbf{s}) \approx \frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} (\hat{\mathbf{w}} + \mathbf{s} - \hat{\mathbf{w}}) + Loss(\hat{\mathbf{w}})$$

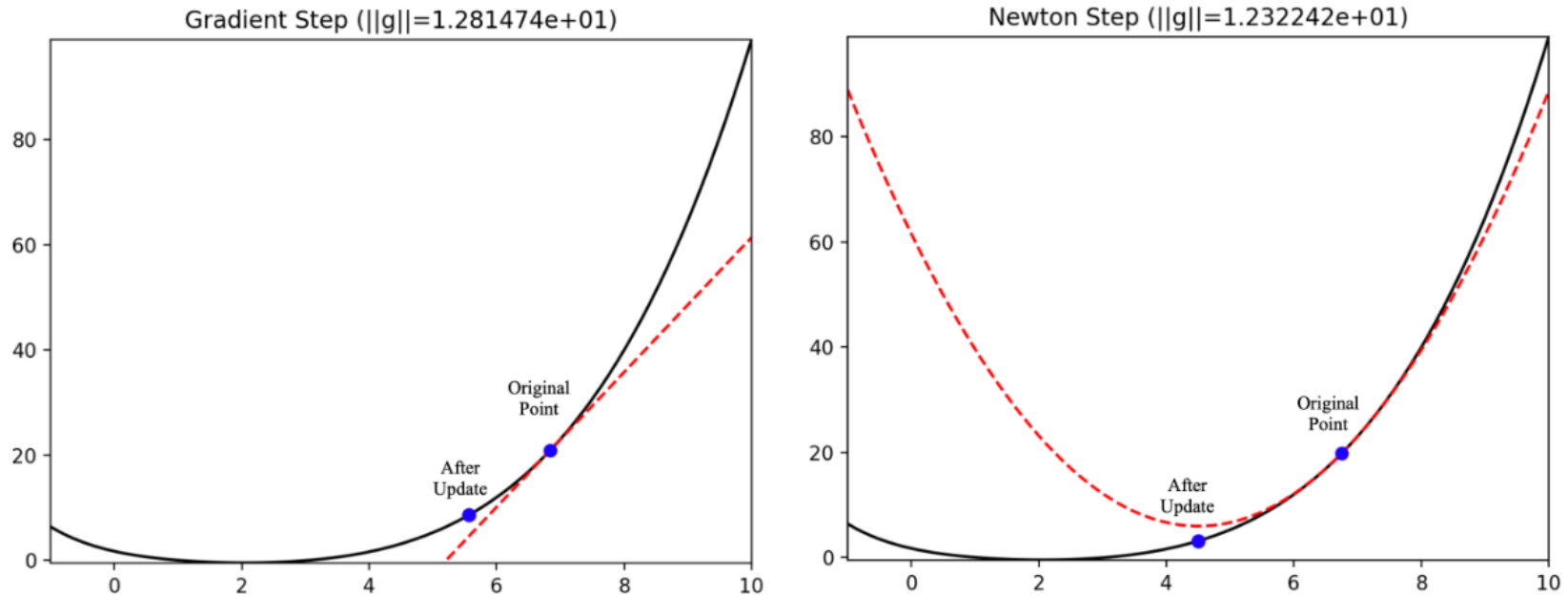


$$Loss(\hat{\mathbf{w}} + \mathbf{s}) \approx \frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} (\mathbf{s}) + Loss(\hat{\mathbf{w}})$$

The goal is to find a vector $\mathbf{S}$ that minimizes this function

$$\mathbf{s} \approx -\alpha \frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} \Rightarrow Loss\left(\hat{\mathbf{w}} - \alpha \frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}}\right) \approx -\alpha \left(\frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} \right)^T \frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} + Loss(\hat{\mathbf{w}}) < Loss(\hat{\mathbf{w}})$$

Solver in Logistic Regression

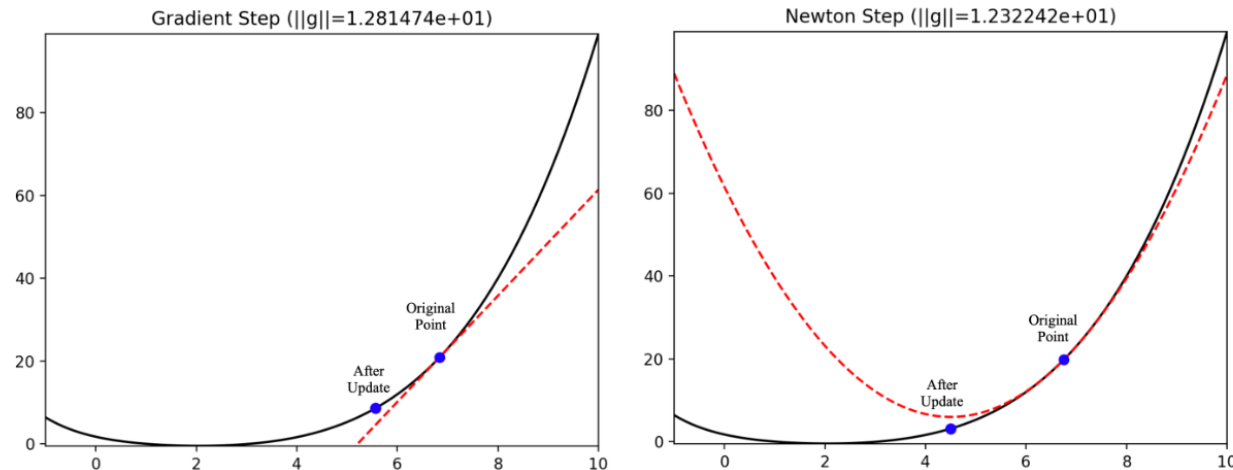


A gradient descent step (left) and a Newton step (right) on the same function. The loss function is depicted in black, the approximation as a dotted red line. The gradient step moves the point downwards along the linear approximation of the function. The Newton step moves the point to the minimum of the parabola, which is used to approximate the function.

- Newton's method assumes that the loss ℓ is twice differentiable and uses the approximation with Hessian (2nd order Taylor approximation).

$$Loss(\hat{\mathbf{w}} + \mathbf{s}) \approx Loss(\hat{\mathbf{w}}) + \left(\frac{\partial Loss(\hat{\mathbf{w}})}{\partial \hat{\mathbf{w}}} \right)^T \mathbf{s} + \frac{1}{2} \mathbf{s}^T H(\hat{\mathbf{w}}) \mathbf{s}$$

Solver in Logistic Regression



- **The gradient descent way:** You look around your feet and no farther than a few meters from your feet. You find the direction that slopes down the most and then walk a few meters in that direction. Then you stop and repeat the process until you can repeat no more. This will eventually lead you to the valley!
- **The Newton way:** You look far away. Specifically you look around in a way such that your line of sight is tangential to the mountain surface where you are. You find the point in your line of sight that is the lowest and using your awesome spiderman powers ...you jump to that point! Then you repeat the process until you can repeat no more!"

Solver in Logistic Regression

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

`solver : {'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'}, default='lbfgs'`

Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects:

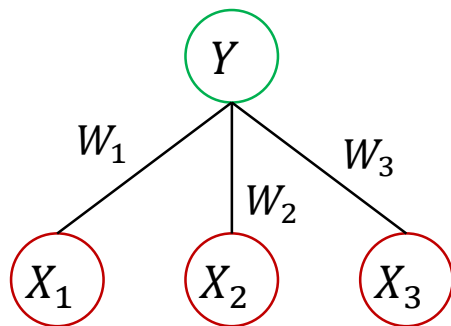
- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- For multiclass problems, only 'newton-cg', 'sag', 'saga' and 'lbfgs' handle multinomial loss;
- 'liblinear' is limited to one-versus-rest schemes.

Gradient descent optimizes a function using knowledge of its derivative. Newton's method, optimizes a function using knowledge of its second derivative. **That can be faster when the second derivative is known and easy to compute (the Newton-Raphson algorithm is used in logistic regression).** **However, the analytic expression for the second derivative is often complicated or intractable, requiring a lot of computation.**

<https://stats.stackexchange.com/questions/253632/why-is-newtons-method-not-widely-used-in-machine-learning>

Summary - Machine Learning $\approx$ Optimization

Perceptron



1. 기계학습 알고리즘 모형 설계 In Supervised Learning

For classification,

$$p(y = 1|x) = \text{Activation}(W_1X_1 + W_2X_2 + W_3X_3 + b)$$

For regression,

$$y = f(x) = W_1X_1 + W_2X_2 + W_3X_3 + W_0 + \varepsilon$$

2. 학습을 위한 평가기준 및 수학적 로직 = 최적화 문제

- **Define Loss function**
- **Max, Min problem**

$\approx$ **Define Optimization Problem**

3. 학습 = 최적 파라미터 = 최적화 문제 풀기

- **문제에 따라 solver 결정 및 적용**
- **Closed form solution = normal eq.**
- **Numerical solution = SVD, Gradient Descent**

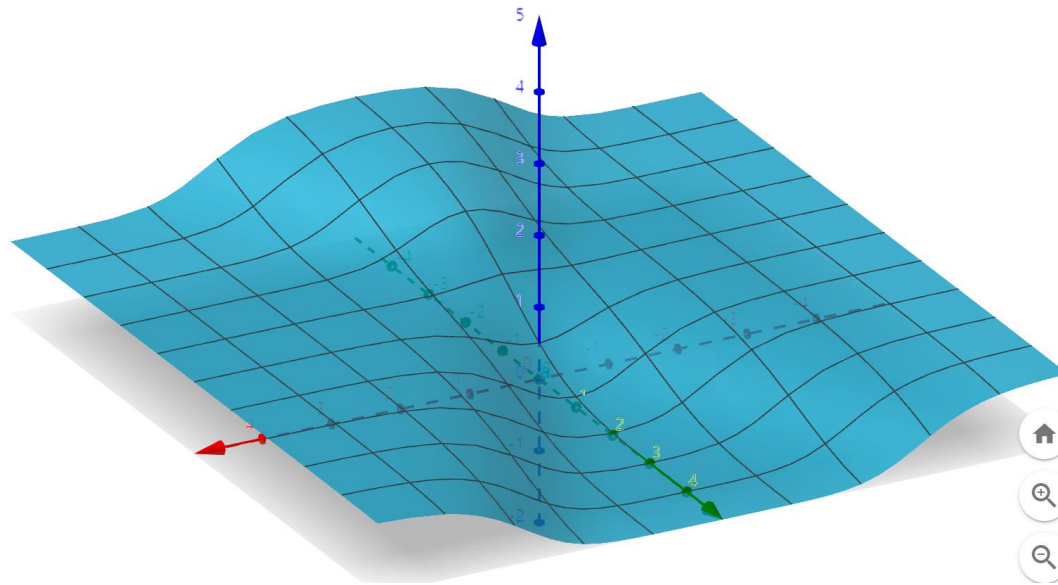
ML 리뷰 요약

- 딥러닝의 기본인 뉴럴네트워크의 가장 단순화 버전은 Perceptron
- 퍼셉트론은 Output 변수의 타입 또는 지도학습 관점에 따라 선형 회귀 모형 또는 로지스틱 회귀 모형의 형태가 동일
- 딥러닝에서의 학습은 주어진 모형의 파라미터를 학습 시키는 것
- 지도학습 관점에 따라 Loss function을 달라 질 수 있다.
- **Loss function을 학습의 평가 기준으로 정하고 이를 통해 파라미터의 적절한 값 찾는 문제는 학문적으로 최적화 문제와 동일하다**
- **결국, 딥러닝을 학습시키는 것은 “딥러닝 알고리즘 관점”에서 최적화 문제를 정의하고 문제를 푸는 것이다**
- 최적화 문제를 푸는 접근법 또한 문제에 따라 다양한 방법론이 존재
- 실제 많은 딥러닝 연구자들을 gradient를 활용한 방법을 적용

ML 리뷰 요약



$$a(x, y) = \left(1 - \frac{1}{1 + e^{-(2x+y)}}\right)^2 + \left(1 - \frac{1}{1 + e^{2x-y}}\right)^2$$



ML 용어

- 지도학습
- 회귀/분류
- 퍼셉트론 – 선형회귀, 로지스틱 회귀 모델
- 분류 모델 - > 결정경계선 발생 Decision Boundary
- 학습 파마리터
- 손실, 비용, 목적 함수
- 잔차제곱 least square, 크로스 엔트로피 Cross-entropy
- 최적화 문제 정의
- Normal Equation – 공분산 Covariance matrix 역수 -> SVD
- 비선형 최적화 문제 => Local search = Gradient Descent
- First-Order / Second-Order